

An LLM-assisted approach to designing software architectures using ADD

Humberto Cervantes, Universidad Autónoma Metropolitana - Iztapalapa, CDMX, Mexico
hcm@xanum.uam.mx

Rick Kazman, Information Technology Management, University of Hawaii at Manoa
Honolulu, Hawaii, USA
kazman@hawaii.edu

Yuanfang Cai, Department of Computer Science, Drexel University
yfcai@drexel.edu

Abstract

Designing effective software architectures is a complex, iterative process that traditionally relies on expert judgment. This paper proposes an approach for Large Language Model (LLM)-assisted software architecture design using the Attribute-Driven Design (ADD) method. By providing an LLM with an explicit description of ADD, an architect persona, and a structured iteration plan, our method guides the LLM to collaboratively produce architecture artifacts with a human architect. We validate the approach through case studies, comparing generated designs against proven solutions and evaluating them with professional architects. Results show that our LLM-assisted ADD process can generate architectures closely aligned with established solutions and partially satisfying architectural drivers, highlighting both the promise and current limitations of using LLMs in architecture design. Our findings emphasize the importance of human oversight and iterative refinement when leveraging LLMs in this domain.

1. Introduction

Software architecture design is a complex process requiring architects to make design decisions that produce structures to satisfy the architectural drivers of the system, quality attributes, primary functional requirements, constraints and other architectural concerns. Recent advances in large language models (LLMs) have led to exploration of AI-assisted tools to perform a number of tasks related to software development, however, their use in architectural design has not yet been explored extensively. The emergence of Generative AI and LLMs has opened new possibilities for facilitating the activity of architectural design. One benefit of LLMs is that they have been trained with vast amounts of information, including documents related to architectural design and catalogs of design patterns and other solutions to design problems.

In this paper we propose an LLM-assisted approach to design software architectures using the Attribute Driven Design method (ADD), a well established method for designing software architectures [Cervantes2024]. Our approach proposes a way to assist software architects in the design process. Our proposal considers several key elements: the LLM is provided with an explicit ADD method description, resulting in a Chain-of-Thought approach.

基于ADD的LLM辅助软件架构设计方法

Humberto Cervantes, 墨西哥自治大都会大学-伊斯塔帕拉帕, CDMX, 墨西哥
hcm@xanum.uam.mx

Rick Kazman, 信息技术管理, 美国夏威夷大学马诺阿分校
kazman@hawaii.edu

德雷克塞尔大学计算机科学系蔡元芳
yfcai@drexel.edu

摘要

设计高效的软件架构是一个复杂且需要反复迭代的过程, 传统上往往依赖专家的主观判断。本文提出了一种基于属性驱动设计 (ADD) 方法的大型语言模型 (LLM) 辅助软件架构设计方案。通过向LLM提供ADD方法的详细说明、架构师角色设定以及结构化迭代计划, 我们的方法能够引导LLM与人类架构师协同生成架构设计成果。我们通过案例研究验证该方法, 将生成的设计方案与成熟解决方案进行对比, 并由专业架构师进行评估。结果显示, 我们的LLM辅助的架构设计流程能够生成与成熟解决方案高度契合的架构, 同时部分满足架构驱动因素, 这既展现了LLM在架构设计中的潜力, 也揭示了其当前的局限性。我们的研究结果强调, 在该领域应用LLM时, 人类监督和迭代优化的重要性。

1. 介绍

软件架构设计是一个复杂的过程, 需要架构师做出设计决策, 构建出既能满足系统架构驱动因素、质量属性、核心功能需求、约束条件及其他架构关注点的结构。近年来, 随着大型语言模型 (LLMs) 的突破性进展, 人们开始探索利用AI辅助工具来完成软件开发相关任务, 但这些工具在架构设计中的应用尚未得到充分探索。生成式AI与LLMs的出现, 为架构设计活动开辟了新可能。LLMs的优势在于其训练过程中积累了海量信息, 包括与架构设计相关的文档资料、设计模式目录以及各类设计问题的解决方案。

本文提出一种基于语言模型 (LLM) 辅助的软件架构设计方法, 采用属性驱动设计 (ADD) 这一成熟的设计方法[Cervantes2024]。该方法旨在为软件架构师提供设计过程中的辅助支持, 其核心要素包括: 向语言模型提供明确的ADD方法描述, 从而形成思维链式设计思路。

We also ask the LLM to plan its design process initially, which can be considered as an instance of Plan-and-Solve prompting [Wang2023]. Furthermore, the LLM makes use of an architect persona [Maranhão2024] and finally, a single centralized architecture document is modified across design iterations. The ADD process is executed step by step, and this results in a collaborative approach between a human and an LLM to design a software architecture and produce solid architecture documentation.

The contributions of our paper include a method for performing ADD using an LLM, and a set of artifacts, including prompts and documents, that are used in conjunction with the process. We evaluate this approach through two case studies, including comparing a generated design with an existing solution and an evaluation of a design by two software architects. We also compare the results of our approach with the results obtained when not providing the LLM with a description of the design process.

To structure our research, we pose the following research questions:

- RQ1. Does an LLM-assisted ADD design result in a solution that is similar to a proven solution?
- RQ2. Does an LLM-assisted ADD design produce architectures that satisfy the architectural drivers?
- RQ3. Does the use of ADD make a significant difference when asking an LLM to design an architecture?

The structure of the paper is as follows: Section 2 presents related work, Section 3 describes the attribute driven design method. Section 4 describes our approach. Section 5 presents the validation and results. Section 6 discusses threats to validity, followed by an analysis and discussion in section 7. Finally, section 8 a conclusion and future work.

2. Related work

The desire to create tools to assist in the architecture design process has existed for a long time. One of the earliest proposals was ArchE [Bachmann2003], a software architecture assistant which was built as a rule based system. Since then, other AI approaches have been explored to assist in the design of software architectures, such as design pattern recommenders [Palma2012]. These approaches have been limited by the challenges associated with the process of designing a software architecture, which is a complex, manual process often reliant on expert knowledge and facing challenges such as time pressure and the difficulty of evaluating multiple architectural options.

The appearance of Large Language Models and Generative AI in recent years is revolutionizing our society and their application in many areas, including software development, is a very active area of research. While AI has significantly impacted other domains of software engineering, its explicit application to SA remains relatively under-explored compared to areas like code generation or testing. Two recent literature reviews [Bucaioni2025] [Esposito2025] have analyzed a number of very recent papers which highlight that using LLMs in architectural design activities is an active area of research.

The work described in [Eisenreich2024] outlines a novel method and tool framework for the semi-automatic generation of software architecture candidates from requirements using large language models (LLMs). Their process involves the automatic generation of domain models and deriving multiple software architecture candidates that are evaluated automatically. [Dhar2025] introduces DRAFT (Domain-specific Retrieval Augmented Few-shot Tuning), a novel approach for generating Architectural Design Decisions (ADDs) by combining the strengths of retrieval-augmented generation (RAG) and fine-tuning of large language models (LLMs). Their approach operates in two phases, an offline phase where a foundational LLM is fine-tuned on a dataset of prompts, and an online phase where the fine-tuned LLM generates design decisions. The authors conclude that DRAFT offers a significant improvement in generating high-quality Design Decisions by effectively combining retrieval-augmented generation

我们还要求大语言模型（LLM）在初始阶段规划其设计流程，这可视为计划-求解式引导的典型示例[Wang2023]。此外，LLM会运用架构师角色模型[Maranhao2024]，最终通过多轮设计迭代对单一集中式架构文档进行修改。ADD流程分步骤执行，这种人机协作模式既实现了软件架构设计，又生成了扎实的架构文档。

本文的贡献包括：提出了一种基于大语言模型（LLM）的ADD方法，以及配套使用的提示词和文档等辅助工具。我们通过两个案例研究验证该方法的有效性：一是将生成的设计方案与现有方案进行对比，二是由两位软件架构师对设计方案进行评估。此外，我们还将本方法的实验结果与未向LLM提供设计流程说明时的对比结果进行了分析。

为构建本研究框架，我们提出以下研究问题：

- RQ1. 基于LLM辅助的ADD设计是否会产生与已验证解决方案相似的结果？
- RQ2. 基于LLM辅助的ADD设计能否生成满足架构驱动因素的架构？
- 研究问题3：在要求法学硕士（LLM）设计架构时，使用需求驱动设计（ADD）是否会产生显著

影响？本文结构安排如下：第2节综述相关研究，第3节阐述属性驱动设计方法，第4节阐述本研究方法，第5节展示验证与结果，第6节讨论效度威胁，第7节进行分析与讨论，最后第8节总结全文并展望未来研究方向。

2. 相关工作

人们长期以来一直渴望开发辅助架构设计的工具。最早提出的方案之一是ArchE[巴赫曼2003]，这款软件架构助手采用基于规则的系统架构设计。此后，学界陆续探索了其他人工智能辅助手段，例如设计模式推荐系统[帕尔马2012]。然而这些方法都受限于软件架构设计的固有挑战——这一过程不仅复杂且需要人工操作，往往依赖专家知识，还面临时间紧迫、难以评估多种架构方案等实际困难。

近年来，大型语言模型（LLM）和生成式AI的涌现正在引发社会变革，其在软件开发等领域的应用已成为研究热点。尽管人工智能已对软件工程其他领域产生深远影响，但相较于代码生成或测试等方向，其在软件架构设计中的具体应用仍处于探索阶段。近期两篇文献综述[布卡奥尼2025][埃斯波西托2025]通过分析大量最新研究成果表明，将LLM应用于架构设计领域正成为研究前沿。

[Eisenreich2024]的研究提出了一种创新方法和工具框架，通过大型语言模型（LLMs）实现需求到软件架构候选方案的半自动生成。该方法包含两个关键步骤：首先自动生成领域模型，随后推导出多个可自动评估的软件架构候选方案。[Dhar2025]提出的“领域特定检索增强少样本调优”（Domain-specific Retrieval Augmented Few-shot Tuning）方法，巧妙融合了检索增强生成（RAG）技术和大型语言模型微调的优势，开创性地开发出架构设计决策（ADDs）生成系统。该方法采用分阶段实施模式：离线阶段通过提示数据集对基础LLM进行微调，线上阶段则由微调后的LLM直接生成设计方案。研究团队指出，这种“检索增强生成”与“微调模型”的协同机制，显著提升了高质量设计方案的生成效果。

with domain-specific fine-tuning. This demonstrates the value of providing the LLM with relevant, domain-specific architectural knowledge. In their work, the authors of [Helmi2025] introduce ARLO, a novel and tailorable approach for automating the transformation of natural language software requirements into software architecture using LLMs and optimization algorithms. The core idea behind ARLO is to first determine the subset of natural language requirements that are architecturally relevant and then map this subset to a customizable matrix of architectural choices. Subsequently, ARLO applies integer linear programming on this architectural-choice matrix to determine the optimal architecture for the given requirements. The interest of this approach is on how they deal with the requirements, as their approach doesn't leverage the LLM to identify architectural choices.

Structuring the interaction with LLMs is another direction. The "Progressive Prompting" method discussed in [Wei2024] involves engaging the LLM in a stepwise manner, following a software development process and generating intermediate artifacts. This approach breaks down complex tasks and provides context. Another structured approach is the use of Prompt Pattern Sequences [Maranhão2024], specifically designed for assisting SA decision-making. The authors describe five prompt patterns designed to address challenges faced by software architects during complex design decisions, these include "Software Architect Persona Pattern", "Architectural Project Context Pattern", "Quality Attribute Question Pattern", "Technical Premises Pattern" and "Uncertain Requirement Statement Pattern". The paper proposes a Prompt Pattern Sequence where these patterns are applied in a specific order to leverage the GPT model effectively. Their approach was evaluated in two real-world scenarios and a fictional case study. The paper highlights that generative AI tools demonstrate significant potential in supporting software architecture decision-making by providing valuable suggestions and speeding up problem resolution.

Furthermore, the potential of AI to facilitate the adoption of design patterns is presented in [Supekar2024]. The authors discuss the problems associated with the lack of use of design patterns, highlighting the complexity and learning curve involved in understanding and implementing them correctly. The paper proposes a novel AI solution that leverages the Azure OpenAI stack and integrates directly within the Visual Studio IDE to provide developers with real-time design pattern recommendations as they code. Their proposed system involves training an LLM using a vast corpus of source code from public repositories.

However, existing work also highlights significant limitations and challenges. LLMs often exhibit irreproducibility of results, a lack of logical similarity between generated outputs for the same query, and an inability to provide complete, holistic solutions [Esposito2025]. In their state of the practice [Jahić2024], the authors also identify several limitations: generated designs can mix abstraction levels and lack clarity in component interactions, LLMs may struggle with identifying what is important in design decisions and require human validation due to potential hallucinations and technical premises that might not fit the project's context. Some authors suggest that while LLMs show potential, they may not yet qualify as reliable engineering techniques for complex design tasks and function more as "inspirational tools".

Our proposed work builds upon the foundation of using LLMs for software architecture design by explicitly providing the LLM with a description of a specific, established design process, Attribute Driven Design (ADD) [Cervantes2024], and the use of something similar to the "Software Architect Persona Pattern" [Maranhão2024] which involves creating a specialized persona incorporating roles, specialties, objectives, and limitations specific to software architecture. Our approach is novel in directly providing the LLM with the methodology of a recognized design process (ADD) itself, alongside with a design plan, a single architecture document and an architect persona. It is worthwhile to mention that we found an undergraduate thesis where an LLM is trained to support architects in the execution of ADD [Rivera2024], however the proposed approach is limited as it does not cover the key design steps of ADD, as we will later discuss.

通过领域特定的微调，这充分证明了为大型语言模型（LLM）提供相关领域架构知识的价值。在[Helmi 2025]的研究中，作者提出了一种名为 ARLO 的创新方法，该方法可灵活定制化地将自然语言软件需求转化为软件架构，其核心在于运用LLM和优化算法实现自动化转换。ARLO 的核心思路是：首先筛选出与架构设计相关的自然语言需求子集，将其映射到可定制的架构选择矩阵；随后通过整数线性规划算法，基于该矩阵确定符合需求的最优架构方案。该方法的独特之处在于其需求处理方式——由于未借助LLM进行架构选择识别，这种处理方式展现出独特的研究价值。

另一个方向是构建与大语言模型（LLM）的交互结构。[Wei2024]中讨论的“渐进式提示法”采用软件开发流程，通过分步骤引导LLM生成中间产物，这种做法能将复杂任务拆解并提供上下文支持。另一种结构化方法是使用[Maranhao2024]提出的提示模式序列，该方法专为辅助系统架构师决策而设计。作者描述了五种针对复杂设计决策中常见挑战的提示模式，包括“软件架构师角色模式”、“架构项目背景模式”、“质量属性提问模式”、“技术前提模式”和“不确定需求陈述模式”。论文提出了一种提示模式序列，通过特定顺序应用这些模式来有效利用GPT模型。该方法已在两个真实场景和一个虚构案例中进行了评估。本文强调，生成式AI工具通过提供有价值的建议和加速问题解决，在支持软件架构决策方面展现出显著潜力。

此外，[Supekar2024]研究揭示了人工智能在促进设计模式应用方面的潜力。作者深入剖析了设计模式应用不足的症结，指出正确理解和实施这些模式存在复杂性和学习曲线。该论文提出了一种创新的AI解决方案，通过整合Azure OpenAI技术栈并直接嵌入Visual Studio集成开发环境，为开发者提供代码编写过程中的实时设计模式建议。其核心系统采用海量公共代码库构建的语料库，训练大型语言模型（LLM）来实现这一功能。

然而现有研究也揭示了重大局限与挑战。大型语言模型常存在结果不可复现、同一查询生成的输出缺乏逻辑关联性、无法提供完整全局解决方案等问题[Esposito2025]。在Jahic2024的最新研究中，作者还指出若干局限：生成的设计方案可能混杂不同抽象层级且组件交互缺乏清晰度，由于存在幻觉现象和技术前提可能与项目背景不符，模型在识别设计决策关键要素时存在困难，需人工验证。部分学者认为，尽管大型语言模型展现出潜力，但其尚未达到复杂设计任务可靠工程工具的标准，更多只能作为“启发式工具”使用。

我们的研究工作以利用大语言模型（LLM）进行软件架构设计为基础，通过向LLM明确提供特定成熟设计流程——属性驱动设计（ADD）[Cervantes2024]的描述，并采用类似“软件架构师角色模型”的框架[Maranhao2024]，即创建包含软件架构专属角色、专长、目标及限制条件的专用角色模型。本方法的创新之处在于直接向LLM提供公认设计流程（ADD）的方法论，同时附带设计计划、单一架构文档和架构师角色模型。值得一提的是，我们发现本科生论文中曾使用LLM辅助架构师执行ADD[Rivera2024]，但该方法存在局限性，因其未涵盖ADD的核心设计步骤，我们将在后文详细讨论。

3. Attribute Driven Design

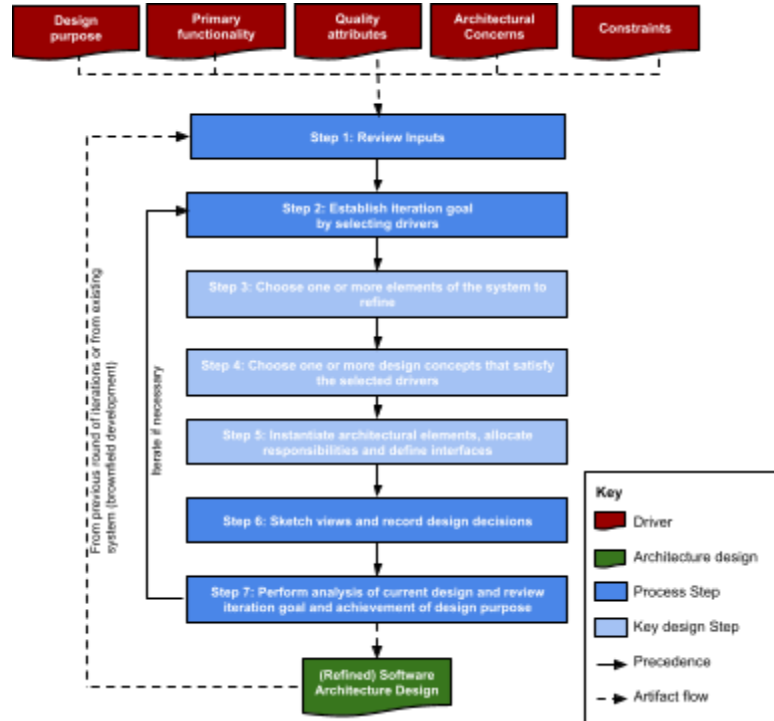


Figure 1 Steps and artifacts of ADD version 3.0

Attribute-Driven Design (ADD) is an architecture design method that provides well defined steps to design an architecture that satisfies architectural drivers [Cervantes2024]. The process is shown in Figure 1, and it consists of the following steps:

- Step 1 focuses on reviewing the architectural drivers (inputs), where the design purpose, the primary functional requirements, quality attributes, constraints and concerns are clarified. Quality attributes are typically described as scenarios that include some metric. Concerns refer to aspects of development that are typically not part of the requirements, but for which design decisions need to be made (e.g. structure the system, log errors).
- Step 2, which is the first step in a design iteration, establishes the goal for the iteration by selecting key design drivers to be addressed.
- Step 3 focuses on the selection of elements in the architecture that must be refined or decomposed to achieve the iteration goals. When a new system is designed, the first iteration considers the system to be the single element to be decomposed.
- Step 4 is where an architect selects various design concepts including reference architectures, design patterns, tactics and externally developed components such as frameworks to provide design solutions for the drivers selected in the iteration goal.
- Step 5 is focused on instantiation of elements. Elements are created from the selected design concepts, relationships between them are established and their responsibilities are allocated. Also, interfaces that allow elements to interact may be defined. By instantiating these elements and connecting them, the structures that compose the architecture are established.

3. 属性驱动设计

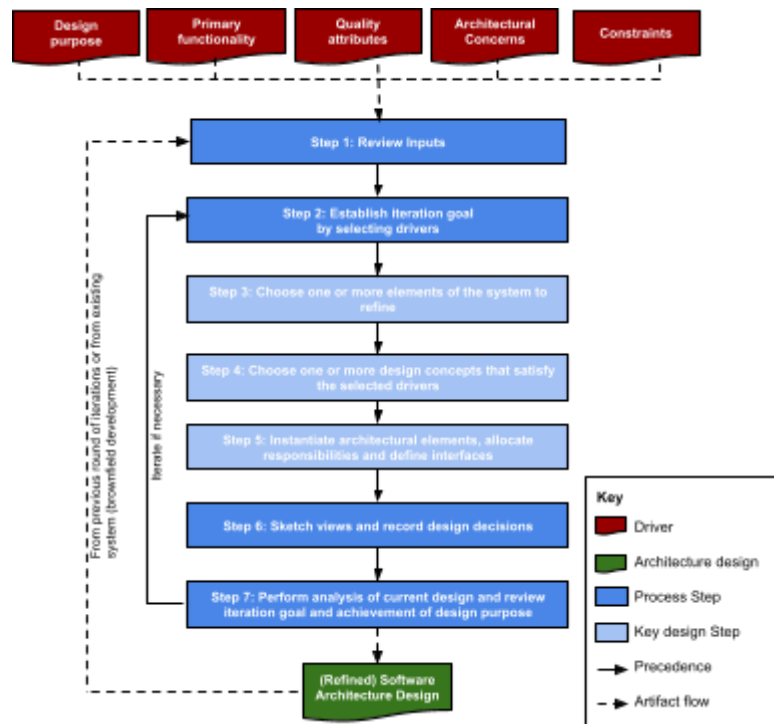


图1 ADD3.0版本的步骤和人工制品

属性驱动设计（ADD）是一种架构设计方法，通过明确的步骤来设计满足架构驱动因素的架构[Cervantes 2024]。该过程如图1所示，包括以下步骤：

- 第一步聚焦于审查架构驱动因素（输入），在此阶段需明确设计目的、主要功能需求、质量属性、约束条件及关注点。质量属性通常被描述为包含特定度量指标的场景。关注点则指开发过程中通常不包含在需求中的方面，但需要做出设计决策（例如系统架构设计、错误日志记录）。
- 步骤2作为设计迭代的第一步，通过选定需解决的关键设计驱动因素来确立迭代目标。
- 第三步聚焦于架构中需细化或分解的元素选择，以实现迭代目标。在新系统设计阶段，首次迭代将系统视为唯一待分解的单元。
- 在第四步中，架构师需从参考架构、设计模式、策略及框架等外部开发组件中筛选多种设计方案，为迭代目标选定的驱动因素提供相应的设计解决方案。
- 第五步聚焦于元素实例化。基于选定的设计概念创建元素，建立其相互关系并分配职责，同时定义允许元素交互的接口。通过实例化这些元素并进行连接，最终构建出架构的组成结构。

- Step 6 focuses on sketching preliminary views and recording of design decisions.
- Step 7 is where an analysis is performed to evaluate if the iteration goal has been achieved and a decision is made on whether or not additional design iterations are needed.

At the end of the iterations, a refined architectural design is produced. It should be noted that this may not be the final design, and that additional design rounds can take place later in the project, these take as input the refined design and start in step 1.

It is important to mention that step 4 of the process, the selection of design concepts to achieve the goal of the iteration, is a particularly difficult step in the design process. As mentioned, design concepts enable architects to avoid reinventing the wheel when designing. Instead, the architect selects solutions that are either conceptual, such as tactics or design patterns, or concrete, such as externally developed components like frameworks or cloud resources. The difficulty lies in the fact that there is a vast number of design concepts to choose from. This can be shown by the extensive lists of pattern catalogs or frameworks that exist today. Another difficult step in the process is step 5, where the previously selected design concepts are instantiated to create structures. The instantiation process involves deciding aspects such as identifying new elements to be created, deciding how a particular pattern is applied or how a selected tactic is achieved, and it can also involve making decisions about a technology, such as deciding on the topics to be used in a messaging platform. In addition to that, correctly connecting the elements and identifying their contracts are fundamental aspects of the instantiation process. The authors of [Rivera2024] acknowledge the difficulty of these steps and decided not to cover them in their proposal, however, we believe that it is precisely in these steps where the use of LLMs provides the most value.

4. Approach

An overview of the approach is presented in figure 2. Our approach is based on the use of an LLM-based coding assistant, as the architecture design process involves working simultaneously with multiple files and, additionally, the design can easily be translated into code later. Figure 2 shows the different artifacts that are used in the process. The left side presents inputs to the process while the right side presents outputs that result from prompting which is guided by a design workflow. We will now detail these elements.

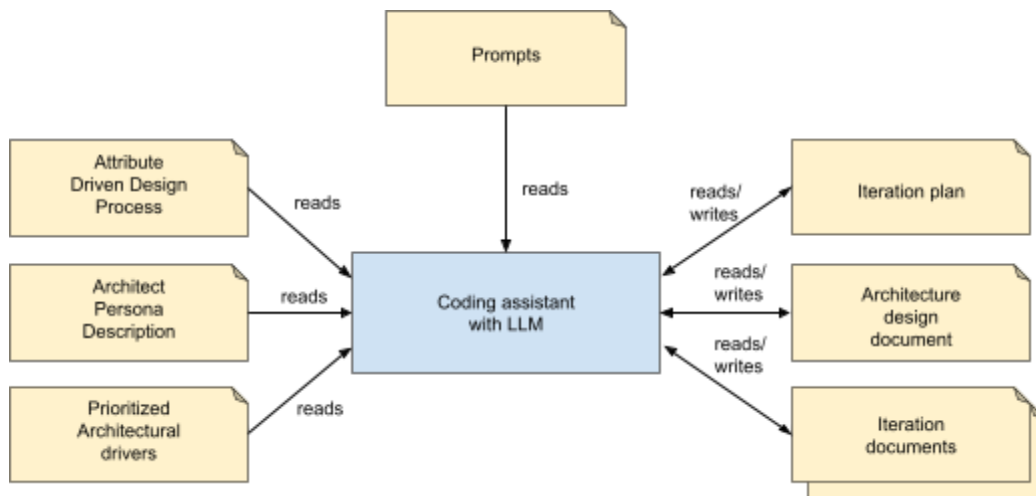


Figure 2. Elements of the approach

- 步骤6着重于绘制初步草图并记录设计决策。
- 步骤7是对迭代目标是否达成进行分析，并据此决定是否需要进行额外的设计迭代。

经过多轮迭代后，最终会形成一个优化后的架构设计方案。需要说明的是，这并非最终定稿，项目后期可能还会进行额外的设计调整，这些调整将以优化后的方案为输入，从第一步重新开始。

需要特别指出的是，流程中的第四步——即为实现迭代目标而筛选设计概念——是整个设计过程中最具挑战性的环节。正如前文所述，设计概念能帮助架构师避免重复造轮子。具体而言，架构师需要从概念性方案（如策略或设计模式）和具体方案（如外部开发的框架或云资源组件）中进行选择。难点在于可供选择的设计概念数量庞大，这一点从当今广泛存在的模式目录或框架清单中可见一斑。流程中的另一个难点出现在第五步，即通过实例化已选设计概念来构建实体结构。该过程需要确定新元素的识别方式、特定模式的应用方法或选定策略的实现方式，同时还需对技术层面做出决策，例如确定消息平台应采用的技术主题。此外，正确连接各元素并识别其合同关系是实例化过程中的关键环节。[Rivera2024]的研究者们承认这些步骤的难度，因此未在提案中涉及，但我们认为，正是在这些环节中，使用大型语言模型（LLMs）能发挥最大价值。

4. 方法

图2展示了该方法的总体架构。我们的方案基于大型语言模型（LLM）编码助手，因为架构设计过程需要同时处理多个文件，且设计成果后续可轻松转化为代码。图2展示了流程中使用的各类组件：左侧展示输入数据，右侧则呈现由设计工作流引导的提示输出结果。接下来我们将详细说明这些组件的具体功能。

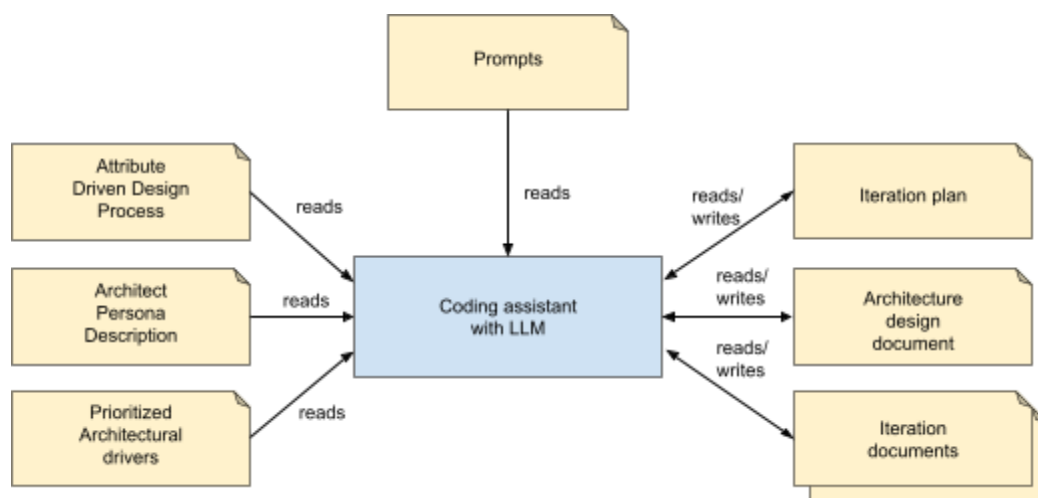


图2. 该方法的组成部分

4.1 Inputs

Our proposed approach considers three different types of inputs.

4.1.1 Attribute driven design process

A novel aspect of our proposal is that we provide a detailed description of the ADD process to the LLM. This technique aligns with Chain-of-Thought or Progressive Prompting methods by breaking down the complex architectural design into sequential steps, generating intermediate artifacts or outputs at each stage [Wei2024]. However, our approach takes one step further by providing a well established process to the LLM. It should be noted that providing this explicit description is essential, as we have observed that LLMs typically hallucinate about the steps of the process when asked about them, so we cannot simply prompt them to perform the steps of ADD from the information learned during training. When the process is executed, as we will later discuss, the process is performed in a step-by-step fashion, allowing the human to review the outcome after each step. The process is described succinctly, and an example of two steps is shown in figure 3. It should be noted that a small adjustment has been made to the original ADD process in that view sketches are produced in step 5 instead of 6. This is because the LLM “felt the need” to create views in step 5 even if not asked to, and this is aligned with what a human would do in step 5, as instantiation typically requires creating diagrams. It should also be noted that mermaid¹ syntax is used for the diagrams, it has been noted that this notation works better than others such as PlantUML². Another adjustment that was made was the extraction of step 1 from the description, this is because step 1 is only performed during the initial iteration and the LLM tends to perform it at every iteration even if the process indicates that it should only be performed in the first iteration.

```
### Step 4: Choose One or More Design Concepts That Satisfy the Selected Drivers
In this step, design concepts are selected to achieve the iteration goal, these
design concepts may include:
- Design patterns: including reference architectures, architectural patterns,
design patterns and deployment patterns
- Externally developed components: including frameworks or specific cloud resources
- Tactics: These are proven strategies to address particular quality attributes

Record selection decisions in a table in the iteration document:

|Selected design concept|Rationale|Discarded Alternatives|
|---|---|---|

### Step 5: Instantiate Architectural Elements, sketch views, allocate
Responsibilities, and Define Interfaces
In this step, the selected design concepts are adapted to address the drivers that
compose the goal of the iteration, that is the meaning of instantiation. This may
result in new elements created or existing elements changed.
```

¹ <https://mermaid.live/>

² <http://plantuml.com/>

4.1 输入

我们提出的方法考虑了三种不同类型的输入。

4.1.1 属性驱动设计过程

我们提案的一个创新点在于，为大型语言模型（LLM）提供了ADD流程的详细描述。该技术通过将复杂的架构设计分解为连续步骤，并在每个阶段生成中间产物或输出，与思维链或渐进式提示方法相契合[Wei 2024]。但我们的方法更进一步，为LLM提供了经过验证的完整流程。需要特别说明的是，这种显性描述至关重要——我们发现当被问及流程步骤时，LLM往往会产生幻觉，因此不能简单地要求它们执行训练中学到的ADD步骤。如后文所述，该流程采用分步执行模式，允许人类在每完成一步后查看结果。流程描述简洁明了，图3展示了两个步骤的示例。需要说明的是，我们对原始ADD流程做了微调：将视图草图的生成步骤从第6步调整到了第5步。这是因为即使未被要求，大语言模型（LLM）在第五步仍“主动”生成了视图，这与人类在第五步的操作逻辑完全一致——毕竟实例化过程通常需要创建图表。需要特别说明的是，这些图表采用了Mermaid¹语法，实践证明这种表示法比PlantUML等工具更具优势²。另一个调整是将第一步操作从描述中单独列出，因为虽然流程说明仅在初始迭代时执行，但大语言模型却会习惯性地每次迭代中重复该步骤。

```
### Step 4: Choose One or More Design Concepts That Satisfy the Selected Drivers
In this step, design concepts are selected to achieve the iteration goal, these
design concepts may include:
  - Design patterns: including reference architectures, architectural patterns,
design patterns and deployment patterns
  - Externally developed components: including frameworks or specific cloud resources
  - Tactics: These are proven strategies to address particular quality attributes

Record selection decisions in a table in the iteration document:

|Selected design concept|Rationale|Discarded Alternatives|
|---|---|---|

### 步骤5: 实例化架构元素、绘制视图、分配职责、定义接口

在此步骤中，所选设计概念需调整以应对构成迭代目标（即实例化意义）的驱动因素。这可能导致新元素的创建
或现有元素的修改。
```

¹ <https://mermaid.live/>

² <http://plantuml.com/>

```
Modify the diagrams in the architecture document according to the instantiation
decisions.
```

```
Record instantiation decisions in a table in the iteration document:
```

```
|Instantiation decision|Rationale|
|---|---|
```

Figure 3 Example steps ADD used to guide the LLM

4.1.2 Architect persona description

As previously discussed, [Maranhão2024] describes the Software Architect Persona Pattern. This ensures that the AI-generated content is accurate and contextually relevant to a software architect's unique roles and constraints. This pattern involves creating a specialized persona incorporating roles, specialties, objectives, and limitations specific to software architecture. Figure 4 shows an excerpt of the software architecture persona description.

```
agent_specification:
  metadata:
    name: Software Architect
    version: 1.0.0
    description: |
      Agent specialized in the design of software architectures, with broad
      experience in different domains
      and cloud platforms.
    domain: Enterprise Systems
    expertise_level: Senior Software Architect

  identity:
    role: Lead Solution Architect
    responsibilities:
      - Analyzing architectural requirements and constraints
      - Designing software architectures for enterprise systems
      - Evaluating different options and trade-offs when making design decisions
      - Documenting the architecture and design decisions
    key_competencies:
      - Architectural design
      - Cloud Architecture (AWS)
      - Microservices Design
      - API Design
      - Security Architecture
      - Data Architecture
      - UML
```

Figure 4 Excerpt of the description of the software architecture persona

4.1.3 Prioritized architectural drivers

Performing the design requires that architectural drivers are provided as inputs to the LLM. These drivers are provided textually and they include:

根据实例化决策对架构文档中的图示进行修改。

Record instantiation decisions in a table in the iteration document:

```
|Instantiation decision|Rationale|
|---|---|
```

图3 ADD用于指导LLM的示例步骤

4.1.2 架构师角色描述

如前所述，[Maranhao2024]提出了软件架构师角色模式。该模式确保人工智能生成的内容准确且符合软件架构师的独特职责与工作限制。其核心在于构建一个包含软件架构专属角色、专业领域、目标及局限性的专业角色模型。图4展示了该软件架构角色描述的节选内容。

```
agent_specification:
  metadata:
    name: Software Architect
    version: 1.0.0
    description: |
      Agent specialized in the design of software architectures, with broad
      experience in different domains
      以及云平台.
    domain: Enterprise Systems
    expertise_level: Senior Software Architect

  identity:
    role: Lead Solution Architect
    responsibilities:
      - Analyzing architectural requirements and constraints
      - Designing software architectures for enterprise systems
      - Evaluating different options and trade-offs when making design decisions
      - Documenting the architecture and design decisions
    key_competencies:
      - Architectural design
      - 云架构 (AWS)
      - 微服务设计
      - API设计
      - 安全体系结构
      - 数据架构
      - UML
```

图4 软件架构角色描述摘录

4.1.3 优先的架构驱动因素

执行该设计要求将架构驱动因素作为输入提供给语言模型（LLM）。这些驱动因素以文本形式提供，包括：

- Primary functional requirements
- Quality attribute scenarios
- Architectural concerns
- Constraints

An important aspect is that primary functional requirements and quality attribute scenarios should be prioritized. Figure 5 shows an example of prioritization, this example will be discussed in section 5.1.

```
##  Priorities

The primary user stories were determined to be:

* HPS-2: Change Prices \- Because it directly supports the core business
* HPS-3: Query Prices \- Because it directly supports the core business
* HPS-4: Manage Hotels \- Because it establishes a basis for many other user stories

The scenarios for the HPS have been prioritized as follows:

| Scenario ID | Importance to the customer | Difficulty of implementation according
to the architect |
| :---- | :---- | :---- |
| QA-1 \- Performance | High | High |
| QA-2 \- Reliability | High | High |
| QA-3 \- Availability | High | High |
| QA-4 \- Scalability | High | High |
| QA-5 \- Security | High | Medium |
| QA-6 \- Modifiability | Medium | Medium |
| QA-7 \- Deployability | Medium | Medium |
| QA-8 \- Monitorability | Medium | Medium |
| QA-9 \- Testability | Medium | Medium |

From this list, QA-1, QA-2, QA-3, QA-4 and QA-5 are selected as primary drivers.
```

Figure 5. Example of prioritization of architectural drivers

4.2. Design workflow

While ADD provides specific steps to perform the actual design of the architecture, a higher level design workflow is needed, as shown in figure 6. This process guides the types of prompts that are used during the design process.

- 主要功能需求
- 质量属性场景
- 建筑问题
- 约束

一个关键点在于，应优先考虑主要功能需求与质量属性场景。
图5展示了一个优先级排序的示例，该示例将在第5.1节中讨论。

```
##  Priorities

The primary user stories were determined to be:

* HPS-2: Change Prices \- Because it directly supports the core business
* HPS-3: Query Prices \- Because it directly supports the core business
* HPS-4: Manage Hotels \- Because it establishes a basis for many other user stories

The scenarios for the HPS have been prioritized as follows:

| Scenario ID | Importance to the customer | Difficulty of implementation according
to the architect |
| :---- | :---- | :---- |
| QA-1 \- Performance | High | High |
| QA-2 \- Reliability | High | High |
| QA-3 \- Availability | High | High |
| QA-4 \- Scalability | High | High |
| QA-5 \- Security | High | Medium |
| QA-6 \- Modifiability | Medium | Medium |
| QA-7 \- Deployability | Medium | Medium |
| QA-8 \- Monitorability | Medium | Medium |
| QA-9 \- Testability | Medium | Medium |

从该列表中，QA-1、QA-2、QA-3、QA-4和QA-5被选为主要驱动因素。
```

图5. 架构驱动因素优先级排序示例

4.2. 设计 workflow

尽管ADD提供了具体步骤以完成架构的实际设计，但仍需采用如图6所示的更高层级设计 workflow。该流程指导设计过程中所使用的提示类型。

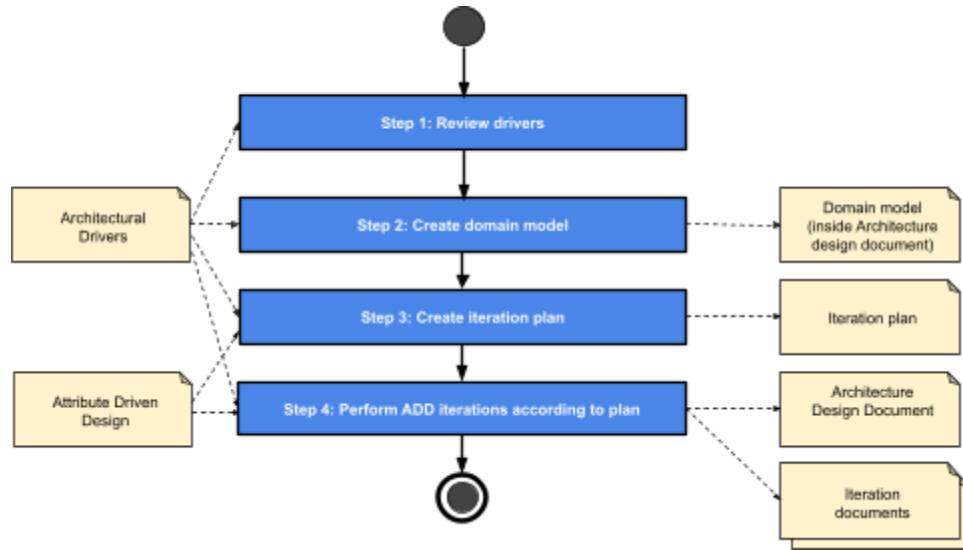


Figure 6 Higher level design process

4.2.1 Step 1. Review drivers

This is the initial step of ADD. Here we simply prompt the LLM to review the requirements and ensure that priorities are understood, as shown in figure 7. Drivers can be modified due to clarification questions.

Please review the ArchitecturalDrivers.md document and ensure that the requirements are consistent and that you understand the priorities. Ask clarification questions, if needed.

Figure 7 Prompt for reviewing drivers

4.2.2 Step 2: Creation of a domain model

The creation of a domain model is typically an activity that occurs during the requirements analysis phase and that precedes design. It is convenient to make this step explicit, to avoid the LLM not performing it initially during the design process. The domain model may be created following a Domain Driven Design (DDD) [Evans2003] approach or a simpler domain model may be chosen, for less complex applications. Figure 8 shows a prompt used to create the domain model.

DDD

Consider the @ArchitecturalDrivers.md and create a domain model for the system using DDD. Create this domain model in a DomainModel.md document in the @Design folder. Represent the domain model using a class diagram using mermaid format. Include a table that describes each element of the domain model and their type (Aggregate Root, Entity, Value Object). Use stereotypes in the class diagram to assign the type of element to the classes.

No DDD

Consider the @ArchitecturalDrivers.md and create a domain model for the system using DDD. Create this domain model in a DomainModel.md document in the @Design

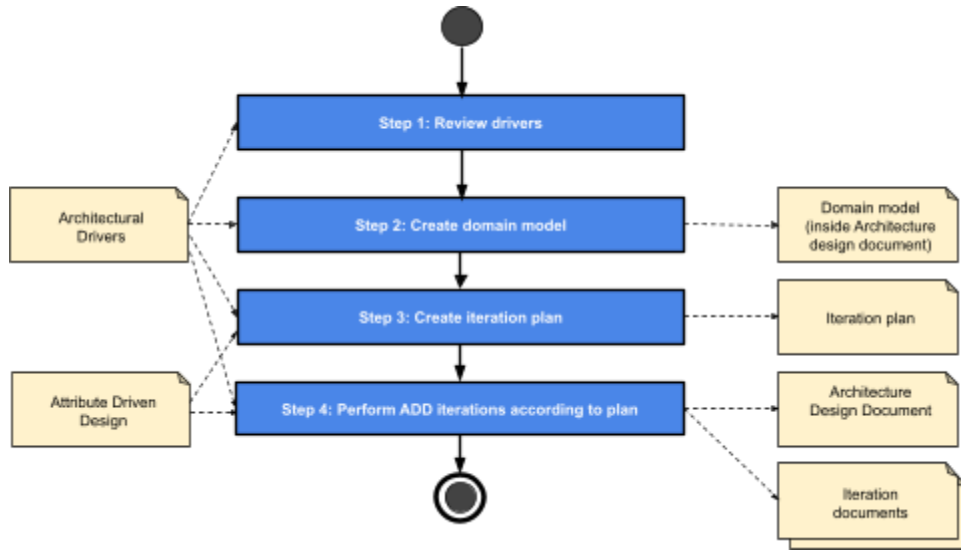


图6 高级设计流程

4.2.1 步骤1. 审查驱动程序

这是ADD的初始步骤。此处我们仅需提示LLM审查需求并确保优先级被理解，如图7所示。驱动因素可能因澄清问题而进行修改。

请参阅《ArchitecturalDrivers.md》文档，确保需求一致且理解优先级。如有疑问，可随时提出澄清问题。

图7 提示 审核驱动程序

4.2.2 步骤2：创建领域模型

领域模型的创建通常是在需求分析阶段进行的活动，且位于设计之前。为避免领域模型语言（LLM）在设计初期未执行该步骤，建议将此步骤明确化。对于复杂度较低的应用，可采用领域驱动设计（DDD）[Evans2003]方法创建领域模型，也可选择更简单的领域模型。图8展示了用于创建领域模型的提示信息。

DD

请参考@ArchitecturalDrivers.md文件，运用领域驱动设计（DDD）方法构建系统领域模型。将该模型创建于@Design文件夹的DomainModel.md文档中，采用Mermaid格式绘制类图进行可视化呈现。需包含描述模型各元素及其类型（聚合根、实体、值对象）的表格，并通过类图中的类型标注功能为对应类别指定元素类型。

无DDD

Consider the @ArchitecturalDrivers.md and create a domain model for the system using DDD. Create this domain model in a DomainModel.md document in the @Design

```
folder. Represent the domain model using a class diagram using mermaid format.
Include a table that describes each element of the domain model.
```

Figure 8 Prompts for creation of a domain model

4.2.3 Step 3: Create an iteration plan

An essential step in guiding the design process is the creation of an iteration plan. This will allow us to know how many iterations to perform as part of the design process. Creating a design plan before actually performing the design activity can be seen as an instance of the Plan-and-Solve prompting [Wang2023] technique, where the LLM is asked to plan and create a roadmap of its work before actually performing it. Figure 9 shows a prompt to create an iteration plan. It should be noted that the creation of an iteration plan requires additional information in the context, in this example the prompt includes references to a vision document, the ADD process and the domain model.

```
Consider the requirement priorities in @ArchitecturalDrivers.md and the domain model
@DomainModel.md. Create an iteration plan for the design of the system, according to
the @AttributeDrivenDesign.md process. Describe, for each iteration, what the main
goal will be. The first iteration should be focused on initially structuring the
system. The following iterations should be focused on addressing high priority
drivers. Be sure to address requirements that directly support the business in early
iterations. Present the iteration plan as a table with iteration number, goal and
list of drivers to be addressed. Output the iteration plan to an IterationPlan.md
file in the @Design folder.
```

Figure 9 Prompts for creation of an iteration plan

4.2.4 Step 4: Architecture document and iteration documents

A key element of this proposal is the use of a single architecture document that centralizes information about the design. This document is read and modified by the LLM across design iterations. This allows the LLM to have a complete overview of the current state of the architecture at every moment and, also, it is a well-structured human readable architectural document. The architecture document may be prepared initially or it can also be created using a prompt such as the one shown in figure 10. The structure of this diagram is partially based in the C4 approach to documenting architectures [Brown2018], but it is enriched with dynamic views and additional information including interfaces and design decisions. This approach was favored over the creation of multiple view documents which are more complicated to handle by the LLM.

```
Create an skeleton of the architecture document in the @Design folder, the
structure of the document is as follows and inside there are instructions of what
you should include for this initial version:
```

1.- Introduction

```
Create a description of the document
```

2.- Context diagram

```
Include the context diagram from the @ArchitecturalDrivers.md document. Include a
paragraph at the beginning that describes what this diagram shows.
```

3.- Architectural drivers

```
Include a summary of the drivers described in @ArchitecturalDrivers.md, including
their priorities. You should separate user stories, quality attribute scenarios,
concerns and constraints in separate tables.
```

4.- Domain model

```
Include the domain model you created in @DomainModel.md
```

文件夹。使用Mermaid格式的类型图表示领域模型，并包含描述领域模型每个元素的表格。

图8 创建域模型的提示

4.2.3 步骤3：制定迭代计划

在设计流程中，制定迭代计划是至关重要的步骤。这能让我们明确需要进行多少次迭代才能完成设计。在实际开展设计工作前制定计划，可以视为应用了“计划-求解”启发式技术[Wang2023]的典型用例——即让大型语言模型在执行任务前预先规划工作路线图。图9展示了创建迭代计划的提示示例。需要特别说明的是，制定迭代计划需要结合更多背景信息，本示例中的提示还包含了对愿景文档、ADD流程和领域模型的引用。

```
Consider the requirement priorities in @ArchitecturalDrivers.md and the domain model @DomainModel.md. Create an iteration plan for the design of the system, according to the @AttributeDrivenDesign.md process. Describe, for each iteration, what the main goal will be. The first iteration should be focused on initially structuring the system. The following iterations should be focused on addressing high priority drivers. Be sure to address requirements that directly support the business in early iterations. Present the iteration plan as a table with iteration number, goal and list of drivers to be addressed. Output the iteration plan to an IterationPlan.md file in the @Design folder.
```

图9 创建迭代计划的提示

4.2.4 步骤4：架构文档与迭代文档

本提案的核心要素在于采用统一架构文档来集中管理设计信息。该文档由LLM在设计迭代过程中持续审阅和修改，既能让LLM随时掌握架构的实时状态，又能形成结构清晰、便于人工阅读的文档。架构文档既可预先编制，也可通过图10所示的提示生成。该图表结构部分基于C4架构文档记录方法[Brown2018]，但新增了动态视图和接口、设计决策等补充信息。相较于需要LLM处理的多视图文档，这种方案更便于管理。

```
Create an skeleton of the architecture document in the @Design folder, the structure of the document is as follows and inside there are instructions of what you should include for this initial version:
```

1.- Introduction

Create a description of the document

2.- Context diagram

Include the context diagram from the @ArchitecturalDrivers.md document. Include a 图表开头的段落说明了该图表所展示的内容。

3 -体系结构驱动因素包括@ ArchitecturalDrivers.md 中描述的驱动因素摘要，包括其优先级。您应该在单独的表中分别列出用户故事、质量属性场景、关注点和约束。

4 -领域模型

在 @ DomainModel.md 中包含创建的域模型

5.- Container diagram

This section contains the main container diagram, according to the C4 approach. Containers include high-level applications or data stores that run within your system, including frontends, databases, message queues, web applications microservices. Create an empty diagram, include a paragraph at the beginning that describes what this diagram is.

This section should also include a table with the name of the container and its responsibilities.

6.- Component diagrams

Only include a paragraph that explains that for each container from the previous section that we will develop, we will include a subsection with a component diagram that will detail the internal design of the container. Each component diagram should have an associated table with the name of the components and their responsibilities.

Don't include anything else in the document right now.

7.- Sequence diagrams

For each use case or quality attribute scenario we will create a sequence diagram. Create a subsection for each of the use case and quality attribute scenario drivers that are mentioned in the @IterationPlan.md.

Include empty sequence diagrams for the moment being.

8.- Interfaces

This section will include details about contracts, leave empty for the moment being.

9.- Design decisions

This section describes the relevant design decisions that resulted in this design.

The section should only include an empty table with the columns driver, decision, rationale and discarded alternative.

Figure 10 Prompt for the creation of an architecture document skeleton

Iteration documents are created as part of the design process, as described in the ADD process (see Figure 3). They summarize aspects of the steps of the ADD process. These documents include some information which is not needed in the architecture document, but which can be useful to understand the thought process during design. This may be a bit analogous to the thought process that is described by reasoning LLMs as they process a prompt. An example of such information is the elements that are being refined during the iteration, the design concepts that are chosen and an analysis at the end of the iteration about the degree with which the drivers have been satisfied.

The actual design process is triggered with a prompt as shown in Figure 11. Notice how we are asking the LLM to pause after each ADD step to allow a human to review and correct if necessary.

Lets proceed with the design using ADD as described in @AttributeDrivenDesign.md. Consider the @IterationPlan.md, the @DomainModel.md and the requirements @ArchitecturalDrivers.md considered to be the goal of this iteration.

Create a document in the @Design folder for the iteration named Iteration1.md.

When the ADD process mentions changes in the iteration document, it is referring to this file you just created. When it mentions changes in the Architecture document, it is referring to @Architecture.md

We will go step by step in the ADD process. At the end of each step you will pause

5.- Container diagram

本节展示C4方法论中的核心容器架构图。容器包含系统运行的高级应用或数据存储单元，涵盖前端界面、数据库、消息队列及Web应用微服务等组件。请创建空白架构图，并在起始段落中说明该图的用途。

本节还应包含一个表格，列出容器名称及其职责。

6.- Component diagrams

Only include a paragraph that explains that for each container from the previous section that we will develop, we will include a subsection with a component diagram that will detail the internal design of the container. Each component diagram should

请创建一个关联表，列出组件名称及其职责。目前文档中请勿添加其他内容。

7.- Sequence diagrams

对于每个用例或质量属性场景，我们将创建一个序列图。对于在@ IterationPlan.md 中提到的每个用例和质量属性场景驱动因素创建一个子章节。

目前暂不包括空序列图。

8.- Interfaces

本节将详细说明合同相关内容，此处暂空。

9.- Design decisions

本节阐述了促成该设计方案的相关设计决策。

该部分仅应包含一个空表格，其列包括驾驶员、决策、依据及废弃方案。

图10 创建体系结构文档骨架的提示

迭代文档是设计流程的重要组成部分，其创建方式与ADD流程（见图3）所述一致。这类文档主要总结ADD流程各步骤的关键要素，其中包含架构文档中无需呈现但有助于理解设计思维过程的信息。这种信息处理方式与语言模型（LLM）处理指令时的推理过程颇为相似。典型示例包括：迭代阶段优化的元素、选定的设计概念，以及迭代结束时对驱动因素满足程度的分析评估。

实际设计流程由提示触发，如图11所示。请注意，我们要求大语言模型（LLM）在每次ADD步骤后暂停，以便人工进行审查并在必要时进行修正。

我们按照@ AttributeDrivenDesign.md 中描述的ADD进行设计。考虑到@ IterationPlan.md、@ DomainModel.md 和需求@ArchitecturalDrivers .md作为本次迭代的目标。

在名为 Iteration1.md 的迭代的@Design文件夹中创建一个文档。

When the ADD process mentions changes in the iteration document, it is referring to this file you just created. When it mentions changes in the Architecture document, it is referring to @Architecture.md

We will go step by step in the ADD process. At the end of each step you will pause

```
and wait for me to review and confirm we can continue.
```

Figure 11 Prompt for performing design

5. Validation and results

To provide a response to our research questions, we performed three experiments with the Cursor IDE³ (v 0.49) using Claude Sonnet 3.7 which at the time of experimentation was highly ranked in programming leaderboards and is also chosen as the default LLM for Cursor⁴. It should be noted that for these experiments, we essentially allowed the LLM to perform the design by confirming the decisions it made at each step, and only few corrections were requested as discussed in section 7. The goal was to understand what would be the “out of the box” solution. As we will discuss later, this is not the ideal approach for performing LLM-assisted design.

5.1 Hotel Pricing System

To answer the first research question RQ1 “Does an LLM-assisted ADD design result in a solution that is similar to a proven solution?”, we perform the design of a documented case study and compare the results with the published solution. In this case, we chose the Hotel Pricing System which is Chapter 8 of the book [Cervantes2024]. This case study was chosen because we have a reference partial solution in the book, so it is possible to compare the design performed using the LLM with the one that was documented in the book and which was used in a real project.

The case study in the book is documented as four iterations with the following goals:

- Iteration 1: Establishing an overall System Structure
- Iteration 2: Identifying structures to support primary functionality
- Iteration 3: Addressing Reliability and Availability quality attributes
- Iteration 4: Addressing development and operations requirements

While this case study is derived from an actual system that was successfully put into production, the case study in the book is presented as a summary of the ADD steps that occur in the four iterations, the goal being to illustrate how the design process is performed. While the most relevant design decisions that were made in the actual system are present in the case study, the order of the iterations is rather oriented to illustrate design decisions focused on particular design aspects. Also, the design in the book does not address all drivers.

For this experiment, we provided the exact same information that is in the book regarding the system requirements. Driver prioritization is described in the book as part of the review of inputs in the first iteration, and that information was also included (see Figure 5). The iteration plan that was proposed by the LLM is shown in table 2 which is a direct copy of the output produced by the LLM.

Iteration	Goal	Drivers to Address
1	Establish overall system structure and deployment model	CRN-1: Establish an overall initial system structure CON-6: Cloud-native approach CON-2: Cloud resource hosting and identity service CRN-5: Set up continuous deployment infrastructure

³ See: <https://www.cursor.com/>

⁴ See: <https://www.anthropic.com/claude/sonnet>

请稍候，待我审核确认后方可继续。

图11 设计执行提示

5. 验证与结果

为解答研究问题，我们使用Cursor集成开发环境³（v0.49）进行了三项实验，实验中采用的Claude Sonnet 3.7模型在当时编程排行榜中名列前茅，同时也是Cursor的默认大语言模型⁴。需要说明的是，在这些实验中，我们基本上让大语言模型自主完成设计——通过确认其在每个步骤的决策，仅如第7节所述提出少量修正建议。实验旨在探究“开箱即用”的解决方案。但正如后文将讨论的，这种做法并非大语言模型辅助设计的理想方式。

5.1 酒店定价体系

为解答第一个研究问题RQ1“基于大语言模型辅助的ADD设计能否产生与成熟方案相似的解决方案？”，我们设计了文档化案例研究，并将结果与已发表的解决方案进行对比。本案例选取了《塞尔万特斯2024》第八章中的酒店定价系统作为研究对象。之所以选择该案例，是因为书中已有参考性部分解决方案，因此可以将基于大语言模型的设计与书中记载的、实际项目中应用的方案进行对比分析。

该书中的案例研究记录为四个迭代阶段，其目标如下：

- 迭代1：建立总体系统结构
- 迭代2：确定支持主要功能的结构
- 迭代3：解决可靠性与可用性质量属性
- 第4轮：解决发展和业务需求

虽然本案例研究源自一个已成功投入生产的实际系统，但书中呈现的案例研究是对四个迭代阶段中ADD步骤的总结，旨在说明设计流程的具体实施方式。尽管案例研究中包含了实际系统中最关键的设计决策，但迭代顺序的编排更侧重于展示针对特定设计维度的决策过程。此外，书中设计案例并未涵盖所有驱动因素。

在本实验中，我们提供了与书中系统要求完全一致的信息。驱动因素优先级排序在书中被描述为首轮迭代输入评审的一部分，该信息亦被纳入（参见图5）。由LLM提出的迭代计划如表2所示，该表直接复制了LLM生成的输出结果。

迭代	球门	驱动程序地址
1	建立整体系统架构和部署模型	CRN -1: 建立初始系统整体结构 CON-6: 云原生方法 CON-2: 云资源托管与身份服务 CRN -5: 建立持续部署基础架构

³ 参见：<https://www.cursor.com/>

⁴ 参见：<https://www.anthropic.com/claude/sonnet>

Iteration	Goal	Drivers to Address
		QA-7: Deployability
2	Design core pricing calculation and publication functionality	HPS-2: Change Prices (Primary user story) QA-1: Performance QA-2: Reliability CON-5: Initial REST API integration with existing systems
3	Design query capabilities and scalability	HPS-3: Query Prices (Primary user story) QA-3: Availability QA-4: Scalability
4	Implement hotel and rate management	HPS-4: Manage Hotels (Primary user story) HPS-5: Manage Rates
5	Design security and user management	HPS-1: Log In HPS-6: Manage Users QA-5: Security CON-1: Multi-platform web interface
6	Enhance modularity and monitoring capabilities	QA-6: Modifiability QA-8: Monitorability QA-9: Testability CRN-4: Avoid technical debt

Table 2. Iteration plan for the HPS

As instructed in the iteration plan creation prompt, the plan starts with an initial structuring of the system, but the rest of the iterations goals were decided by the LLM. It is interesting to observe that the iterations in this plan combine the satisfaction of functional requirements and quality attributes, and that the LLM correctly selects quality attributes that are associated with the functionality.

The details of the design are too extensive to be included in this paper, but they can be found in the following location: <https://github.com/otrebmuH/HotelPricingSystem>. A comparison of several aspects of both solutions is done across the following dimensions and the results can be found in appendix A.

- System structure.
- Components.
- Sequence diagrams.
- Contracts.
- Communication style.
- Domain modeling.
- API management.
- Security.
- Scalability.
- Deployability.

迭代	球门	驱动因素
		QA-7: 可部署性
2	设计核心计算与出版功能	HPS -2: 变更价格（主要用户故事） QA-1: 性能 QA-2: 可靠性 CON-5: 与现有系统的初步REST API集成
3	设计查询能力和可扩展性	HPS -3: 查询价格（主要用户故事） QA-3: 可用性 QA-4: 可扩展性
4	实施酒店及费率管理	HPS -4: 酒店管理（主要用户故事） HPS -5: 管理费率
5	设计安全与用户管理	HPS -1: 登录 HPS -6: 管理用户 QA-5: 安全 CON-1: 多平台网页界面
6	增强模块化和监测能力	QA-6: 可修改性 QA-8: 可监控性 QA-9: 可测试性 CRN -4: 避免技术债务

表2. HPS 的迭代计划

根据迭代计划创建提示中的说明，该计划以系统初始架构设计为起点，其余迭代目标均由语言模型（LLM）确定。值得注意的是，该计划中的迭代既满足功能需求又兼顾质量属性，且语言模型能准确选择与功能相关的质量属性。

设计的细节过于繁杂，无法在本文中列出，但可以在以下位置找到：<https://github.com/otrebmuH/HotelPricingSystem>。以下维度对两种解决方案的几个方面进行了比较，结果可以在附录A中找到。

- 系统结构。
- 组件。
- 序列图
- 合同。
- 沟通方式。
- 领域建模。
- API管理。
- 保护措施
- 可扩展性。
- 可部署性。

- Testability.

We can now provide an answer to RQ1. We can observe that both the case study in the book and the LLM generated solution are relatively similar with respect to their overarching principles. Similar design decisions were made, such as the use of CQRS and deploying the system as microservices, however, these are very common decisions in today's systems. The scope of the case study in the book is limited in comparison to the design generated by the LLM (for instance, not all drivers are addressed in the book's case study) but this can be explained by the fact that the book's purpose was to illustrate the design process and not to be a complete architectural document for the HPS system. It should be noted that the real HPS system that was deployed to production included several of the aspects that were proposed in the LLM-based solution.

5.3 Event ticketing system

To answer the third research question RQ2 “Does an LLM-assisted ADD design produce architectures that satisfy the architectural drivers?”, we perform the design of a new and unpublished case study. This case study is focused on the design of an event ticketing system. Evaluation is performed using a scenario-based approach, similar to the “Lightweight ATAM” [Bass2021], with two professional software architects.

The event ticketing system is a case study which was specifically created to teach this approach and, as such, we are certain that the LLM did not see the exact requirements documentation during its training. It should be noted, though, that examples of similar types of systems exist on the internet (see section 6). This is a system similar to the ones used to buy concert tickets and was selected because understanding of the problem is relatively simple as a large number of people have had the opportunity to interact with this type of system. It also poses interesting challenges regarding scalability and handling large volumes of users buying tickets simultaneously (as it happens with major events today).

A vision document, list of quality attribute scenarios, concerns and priorities for this system can be found in: <https://github.com/otrebmu/EventTicketSystem>. This information, and additional more detailed user stories were provided to the LLM for the design process. The user stories were documented in individual files. The iteration plan that was proposed by the LLM is shown in table 3 (concerns were also part of the drivers but are omitted to reduce space). it should be noted that for this system, we asked the LLM to create the iteration plan specifically for high priority requirements, to avoid the creation of a very large design document.

Iteration	Goal	Drivers to Address
1	Establish the fundamental system structure and implement integration with the external Identity Provider.	User Stories: • US001: User Registration (via IdP) • US002: Email Verification (via IdP) • US003: User Login (via IdP) • US004: Password Reset (via IdP) Quality Attribute Scenarios: • QAS004: Authentication Security • QAS022: Environment Consistency • QAS014: External Service Integration

- 可测试性。

我们现在可以回答RQ1。我们可以观察到，书中的案例研究和由大型语言模型生成的解决方案在总体原则方面相对相似。两者都做出了类似的设计决策，例如使用CQRS和将系统部署为微服务，然而，这些是当今系统中非常常见的决策。与大型语言模型生成的设计相比，书中的案例研究范围有限（例如，书中案例研究并未涵盖所有驱动因素），但这一点可以通过书的目的来解释，即展示设计过程而非作为 HPS 系统的完整架构文档。值得注意的是，实际部署到生产环境的 HPS 系统包含了基于大型语言模型解决方案中提出的多个方面。

5.3 事件售票系统

为了回答第三个研究问题 RQ2 “LLM 辅助的 ADD 设计是否能产生满足架构驱动因素的架构？” 我们设计了一个新的未发表案例研究。该案例研究专注于事件票务系统的设计。评估采用基于场景的方法，类似于“轻量级 ATAM” [Bass2021]，由两位专业软件架构师完成。

本次活动的票务系统作为教学案例专门开发，因此我们确信法学硕士生在培训期间并未接触到完整的需求文档。不过需要说明的是，类似系统在互联网上已有现成范例（详见第六节）。该系统与音乐会购票平台类似，之所以被选中是因为其问题理解相对简单——毕竟多数人都有过使用这类系统的经验。同时，该系统在可扩展性方面也颇具挑战性，需要应对大规模用户同时购票的场景（这正是当今大型活动的常见情况）。

关于该系统的愿景文档、质量属性场景清单、关注点及优先级可查阅<https://github.com/otrebmuh/EventTicketSystem>。这些信息以及更详细的用户故事已提交给LLM参与设计流程。用户故事被整理在独立文件中。LLM提出的迭代计划详见表3（关注点虽也是驱动因素，但为节省篇幅已省略）。需要说明的是，针对该系统，我们特别要求LLM针对高优先级需求制定迭代计划，以避免生成过长的设计文档。

迭代	球门	驱动程序地址
1	构建基础系统架构，并实现与外部身份提供方的集成。	用户故事： ● US001：用户注册（通过身份证明机构） ● US002：电子邮件验证（通过身份验证机构） ● US003：用户登录（通过身份验证服务器） ● US004：密码重置（通过身份验证服务器） 质量属性场景： ● QAS004：身份验证安全 ● QAS022：环境一致性 ● QAS014：外部服务集成

Iteration	Goal	Drivers to Address
2	Implement core event management capabilities and basic search functionality.	User Stories: • US005: Event Creation • US007: Event Listing Display • US018: Basic Event Search Quality Attribute Scenarios: • QAS002: Search Response Time • QAS009: Mobile Responsiveness
3	Implement ticket inventory management and basic order processing capabilities.	User Stories: • US009: Inventory Creation • US010: Inventory Updates • US012: Ticket Selection • US014: Order Confirmation Quality Attribute Scenarios: • QAS001: High-Concurrency Ticket Purchase • QAS007: Transaction Integrity • QAS008: Data Consistency • QAS013: Payment Gateway Integration
4	Implement secure payment processing and ticket generation capabilities.	User Stories: • US013: Payment Processing • US015: Ticket Generation • US016: Ticket Delivery Quality Attribute Scenarios: • QAS003: Payment Data Protection • QAS006: Database Scaling • QAS015: System Recovery from Failure

Table 3. Iteration plan for the Event Ticketing System

The design of the event ticket system is also documented in a very comprehensive architecture document which can be found in <https://github.com/otrebmu/EventTicketSystem/blob/master/Design/Architecture.md>

To evaluate this solution, we conducted a facilitated lightweight scenario-based evaluation with two experienced architects from industry. The architecture document and requirements information was provided beforehand. The evaluation was performed in a 1.5 hour meeting with the following agenda:

1. Case study presentation
2. Overview of the solution (container diagram)
3. Scenario review (using sequence diagrams)
4. Discussion and closing

Table 4 presents the drivers that were reviewed during the evaluation, a short summary of how they are addressed and the opinion of the architects on whether or not the design satisfied the driver completely, partially or not.

Driver	How it is addressed	Architect 1	Architect 2
--------	---------------------	-------------	-------------

迭代	球门	驱动因素
2	实现核心事件管理功能及基础搜索功能。	用户故事： ● US005: 事件创建 ● US007: 事件列表显示 ● US018: 基本事件检索 质量属性场景： ● QAS002: 搜索响应时间 ● QAS009: 移动响应性
3	实现工单库存管理及基础订单处理功能。	用户故事： ● US009: 库存创建 ● US010: 库存更新 ● US012: 票务选择 ● US014: 订单确认 质量属性场景： ● QAS001: 高并发购票 ● QAS007: 事务完整性 ● QAS008: 数据一致性 ● QAS013: 支付网关集成
4	实现安全的支付处理和票据生成功能。	用户故事： ● US013: 支付处理 ● US015: 生成票据 ● US016: 票务交付 质量属性场景： ● QAS003: 支付数据保护 ● QAS006: 数据库扩展 ● QAS015: 系统故障恢复

表3 事件票务系统迭代计划

活动票务系统的设计也记录在一份非常全面的架构文档中，可以在<https://github.com/otrebmu/EventTicketSystem/blob/master/Design/Architecture.md>找到

为评估该解决方案，我们邀请了两位行业资深架构师，通过轻量级场景化评估方案展开协作。评估前已提供完整的架构文档及需求信息，整个过程在1.5小时的会议中完成，议程如下：

1. 案例研究展示
2. 解决方案概述（容器图）
3. 情景回顾（使用序列图）
4. 讨论与结语

表4列出了评估过程中审查的驱动因素、其处理方式的简要概述，以及建筑师对设计是否完全、部分或未完全满足该驱动因素的意见。

司机	如何解决	建筑师1	建筑师2
----	------	------	------

US003: User Login	• OAuth2 + SSO via IdP: The system authenticates users through the external Identity Provider using OAuth2, effectively implementing single sign-on. • JWT Token Authentication: After IdP authentication, the Auth Service issues JWT tokens for use by other services, enabling stateless, cross-service authorization. • Session Management via Cache: Instead of server sessions, a distributed cache (Redis) is used for any session state.	Partial	Yes
US012: Ticket Selection	• Inventory Reservation System: To handle simultaneous ticket selection by many users, the architecture implements a Reservation System for tickets. When a user selects tickets, those are reserved to prevent others from taking them, addressing oversell issues. • Optimistic Locking & Consistency: Optimistic locking on inventory ensures that multiple selections can't reduce inventory below zero. Also, eventual consistency is embraced to boost performance under high concurrency. • Connection Pooling: The system uses database connection pooling to handle the burst of many ticket selection transactions without exhausting resources.	Yes	Yes
US013: Payment Processing	• Payment Service Microservice: A dedicated Payment Service handles all payment processing, isolating financial transactions from other logic. • PaymentGateway Adapter Pattern: The design explicitly introduces a PaymentGatewayAdapter pattern. This adapter abstracts the integration with external payment providers, allowing multiple gateways and providing a unified interface. It also handles gateway errors. • Payment Security Managers: Strong security components (PaymentSecurityManager and PaymentKeyManager) are part of the Payment Service, ensuring sensitive card data is encrypted, masked, and keys are safely managed. Circuit breakers are also applied to external calls to handle gateway downtime.	Yes	Yes
US014: Order Confirmation	• Order Service & Saga Pattern: The Order Service orchestrates order finalization, and the architecture applies the Saga pattern for distributed transaction integrity. This ensures that inventory reservation, payment, and ticket generation either all succeed or are rolled back on failure. • Outbox Pattern for Events: The system uses an Outbox pattern to reliably send events/messages (e.g., to trigger ticket generation) once an order is confirmed. This guarantees that confirmation events aren't lost even if a service crashes at that moment. • Order Validation & CQRS: Before confirmation, the Order Service validates the order (all pieces are consistent) and uses CQRS – separating read vs. write models – to optimize performance of confirmation and later queries.	Partial	Partial
US015: Ticket Generation	• Ticket Service Microservice: A dedicated Ticket Service is responsible for generating ticket artifacts (QR codes, PDFs, etc.). This separation means ticket creation can be managed and scaled independently of orders. • Factory & Builder Design Patterns: The architecture chooses to implement the Ticket Service using Factory/Builder patterns. This allows support for multiple ticket formats and easy extension to new formats by encapsulating creation logic per type. • Ticket Security & Recovery Components: The TicketService design includes a TicketSecurityManager for applying security features to tickets and a TicketRecoveryManager for handling failures. These ensure tickets are tamper-resistant and that ticket generation can recover from crashes (so tickets aren't lost).	Partial	Partial
US016: Ticket	• Delivery Service Microservice: A separate Delivery Service handles	Partial	Partial

US003: 用户登录	• OAuth2 + SSO 通过IdP: 系统通过OAuth2使用外部身份提供程序对用户进行身份验证, 有效地实现了单点登录。 • JWT Token身份验证: IdP身份验证后, Auth Service会为其服务签发 JWT 令牌, 支持无状态跨服务授权。 • 通过缓存管理会话: 采用分布式缓存 (Redis) 来管理会话状态, 而非使用服务器会话。	部分	是
US012: 票务选择	• 库存预订系统: 为解决多用户同时选票的问题, 系统架构中设计了票务预订机制。当用户完成选票操作后, 系统会立即完成预订, 防止其他用户抢购, 从而有效避免超额销售。 • 乐观锁与一致性: 库存的乐观锁机制确保多个并发操作无法将库存量降至零以下。同时, 系统采用最终一致性原则, 从而在高并发场景下提升性能。 • 连接池: 系统采用数据库连接池, 处理大量选票事务的突发, 不耗尽资源。	是	是
US013: 支付处理	• 支付服务微服务: 一个专用的支付服务处理所有的支付处理, 将金融事务与其他逻辑隔离。 • PaymentGatewayAdapter模式: 该设计明确引入了PaymentGatewayAdapter模式。该适配器抽象了与外部支付提供商的集成, 允许使用多个网关并提供统一接口。它还处理网关错误。 • 支付安全管理器: 强大的安全组件 (PaymentSecurityManager和PaymentKeyManager) 是支付服务的一部分, 可确保加密敏感的卡片数据、屏蔽敏感数据, 并安全地管理密钥。还应用断路器处理外部调用, 以处理网关停机。	是	是
US014: 订单确认	• 订单服务与Saga模式: 订单服务负责协调订单最终确认, 系统架构采用Saga模式来保障分布式事务的完整性。该机制确保库存预留、支付及工单生成这三个环节要么全部成功, 要么在失败时自动回滚。 • 事件的Outbox模式: 系统使用Outbox模式在订单确认后可靠地发送事件/消息 (例如, 触发工单生成)。这保证了即使服务在那一刻崩溃, 确认事件也不会丢失。 • 订单验证和 CQRS: 在确认之前, 订单服务验证订单 (所有部件都一致), 并使用 CQRS —— 分离读取和写入模型 —— 来优化确认和后续查询的性能。	部分	部分
US015: 生成票据	• 门票服务微服务: 一个专用的门票服务负责生成门票工件 (二维码、pdf等)。这种分离意味着门票创建可以独立于订单进行管理和扩展。 • Factory & Builder设计模式: 该架构选择使用Factory/Builder模式实现Ticket Service。这允许支持多种票证格式, 并通过按类型封装创建逻辑, 轻松扩展到新格式。 • 门票安全和恢复组件: 门票服务设计包括门票安全管理器, 用于对门票应用安全功能, 以及门票恢复管理器, 用于处理故障。这些组件确保门票防篡改, 并且门票生成可以从崩溃中恢复 (因此门票不会丢失)。	部分	部分
US016: 票	• 交付服务微服务: 一个独立的交付服务处理	部分	部分

Delivery	sending tickets via various channels (email, SMS, etc.) and tracking delivery status. This decouples delivery from ticket creation. • Strategy Pattern for Delivery Channels: The architecture uses a Strategy pattern in the Delivery Service. This means different delivery methods (email attachment, SMS link, mobile app) are implemented as interchangeable strategies, allowing easy addition of new methods and graceful handling of each. • Delivery Failure Handling: The Delivery Service is designed to handle failures (e.g., email bounce) gracefully, and publishes delivery events. A Notification Service subscribes to delivery results, which can trigger user notifications or retries. Automatic failover and health checks ensure high availability of the delivery.		
QAS001: High-Concurrency Ticket Purchase	• Optimistic Locking on Inventory: This ensures minimal contention – transactions fail and retry if inventory changed, preventing double-sells while keeping throughput high. • Ticket Reservation System: The system implements a reservation hold for tickets, so when one user selects tickets, they are temporarily unavailable to others. This further prevents overselling under extreme load. • Connection Pooling: Use of a connection pool for the database means the system can handle a spike of DB operations without running out of connections, thereby serving many concurrent buyers efficiently.	Partial	Partial
QAS015: System Recovery from Failure	• RecoveryManager Components: The architecture includes RecoveryManager components in services to handle failover and backups. These components manage backup replication and initiate failover if a service instance or region fails. • Multi-Region Deployment with Automated Failover: A key decision is to deploy across multiple regions with automated failover mechanisms. This means if an entire data center or region goes down, a secondary region takes over quickly. Health checks and orchestrators switch traffic to the backup site with minimal intervention. • Frequent Backups and Snapshots: The system takes regular snapshots of critical data (e.g., Inventory snapshots) and has backup strategies in place for each service's data. These ensure that after a crash, data can be restored to a recent point (meeting RPO requirements), and services can resume.	No	No

Table 4. Drivers reviewed during the scenario based-evaluation

The information in table 4 shows that the evaluators were not completely convinced that the presented design completely satisfied the driver in several cases. One of the recurrent problems that was observed is that the sequence diagrams show that all use case scenarios (such as ticket confirmation or ticket delivery) must be triggered from the user interface, while it would make more sense that they are triggered by the reception of an event or a periodic process. The problems in the sequence diagrams may be because the LLM did not consider the use case detailed description, so it simply decided these use cases were similar to others.

分娩	通过多种渠道（如电子邮件、短信等）发送工单并追踪配送状态，从而将配送与工单创建分离。 • 配送渠道策略模式：该架构在配送服务中采用策略模式。这意味着不同的配送方式（电子邮件附件、短信链接、移动应用）被实现为可互换的策略，便于轻松添加新方法并优雅处理每种方式。 • 递送失败处理：递送服务设计用于优雅地处理失败（例如电子邮件退信），并发布递送事件。通知服务订阅递送结果，可触发用户通知或重试。自动故障转移和运行状况检查确保递送的高可用性。		
质量保证体系001： 高并发性 a 票务购买	• 库存乐观锁：该机制可将并发冲突降至最低——当库存发生变化时，事务将失败并重试，从而防止重复销售，同时保持高吞吐量。 • 票务预订系统：系统实现票务预订保留，所以当用户选择票务时，其他用户暂时无法使用。这进一步防止了在极端负载下超额销售。 • 连接池：使用数据库连接池意味着系统可以处理DB操作的峰值而不耗尽连接，从而高效地为许多并发买家服务。	部分	部分
QAS015：系统 故障恢复	• RecoveryManager组件：该体系结构包括在服务中处理故障转移和备份的RecoveryManager组件。这些组件管理备份复制，并在服务实例或区域发生故障时启动故障转移。 • 基于自动故障转移的多区域部署：关键决策是采用多区域部署架构并配备自动故障转移机制。这意味着当整个数据中心或某个区域发生故障时，备用区域将立即接管。通过健康检查和编排器的协同工作，流量可在最小干预下无缝切换至备用站点。 • 频繁的备份和快照：系统定期对关键数据（如库存快照）进行快照，并为每个服务的数据制定了备份策略。这些措施确保在系统崩溃后，数据可以恢复到最近的时间点（满足 RPO 要求），并使服务能够继续运行。	不	不

表4. 基于情景的评估中审查的驱动因素

表4的数据表明，评估人员在多个案例中对设计方案是否完全满足需求仍存疑虑。反复出现的问题是：序列图显示所有用例场景（如工单确认或工单交付）都必须通过用户界面触发，但更合理的触发方式应该是事件接收或周期性流程。序列图中的问题可能源于LLM模型未充分考虑用例的详细描述，因此简单地将这些用例判定为与其他用例相似。

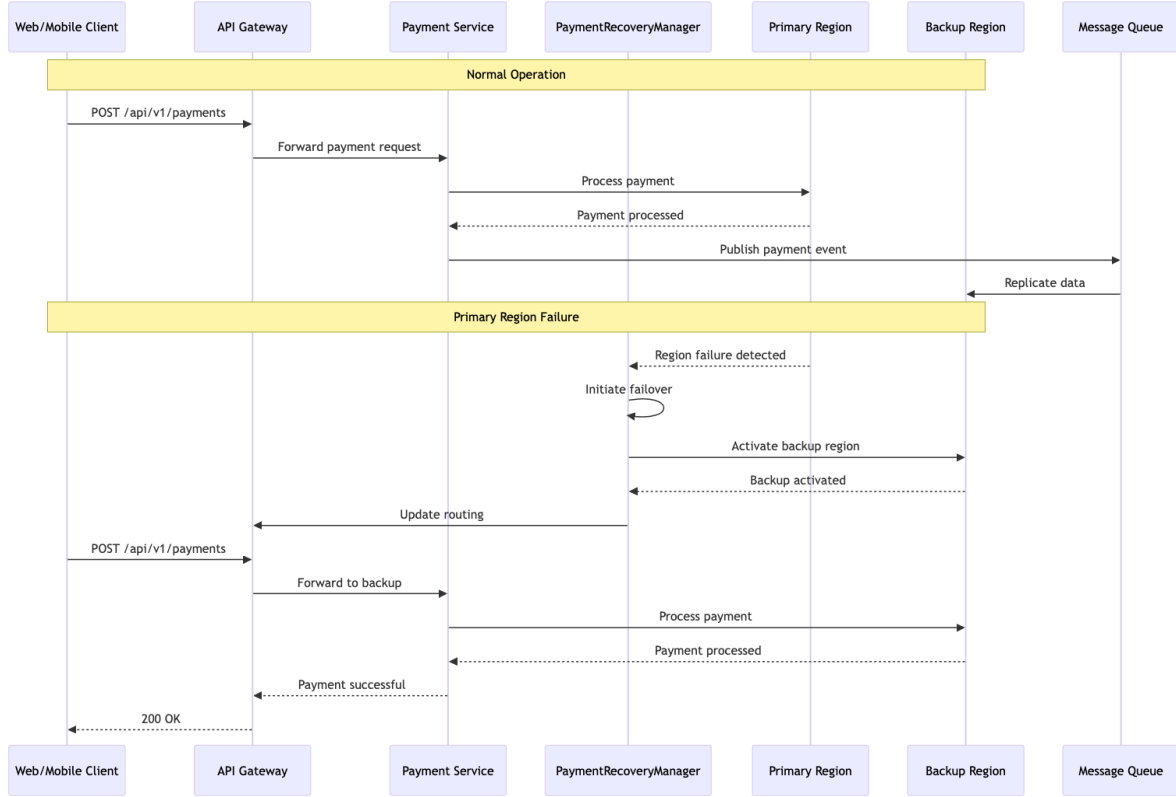


Figure 12 Sequence diagram for QAS015

In the case of QAS015 (see Figure 12) a PaymentRecoveryManager initiates a failover, activates a backup region and also requests routes to be updated. This component does not appear elsewhere in the document, but there are other similar components such as TicketRecoveryManager or DeliveryRecoveryManager and a design decision is documented: “Implement RecoveryManager components with failover and backup strategies”. This is an example of a discrepancy in the design. Also, the proposed solution was not accepted because it seems like a component within a service is responsible for managing recovery, while this should be a more global approach which is not the responsibility of each service. Although the evaluators observed issues in the design, they mentioned they were impressed by the architecture document and thought this approach could be very useful to practitioners.

We can now provide an answer to RQ2: In our experiment, the LLM produced an architecture design that partially satisfied the drivers. However, in this experiment, the architect did not request the LLM to perform many corrections on the design that is produced. We believed that a careful review and correction of the outputs as they are produced would have resulted in a design that better satisfied the architectural drivers.

5.3 Design without ADD

To answer the third research question RQ3 “Does the use of ADD make a significant difference when asking an LLM to design an architecture?”, we asked the LLM to perform design without providing the ADD process and associated iteration plan. To explore the extent to which ADD influenced the design outputs we ran three experiments, summarized in table 5. The first experiment employed a zero-shot approach; in the second experiment, we provided an empty architecture document template to the LLM; in the third experiment, we provided this architecture document template and also provided additional instructions in the template such as: “For each use case and quality attribute scenario, include a sequence diagram that shows how the

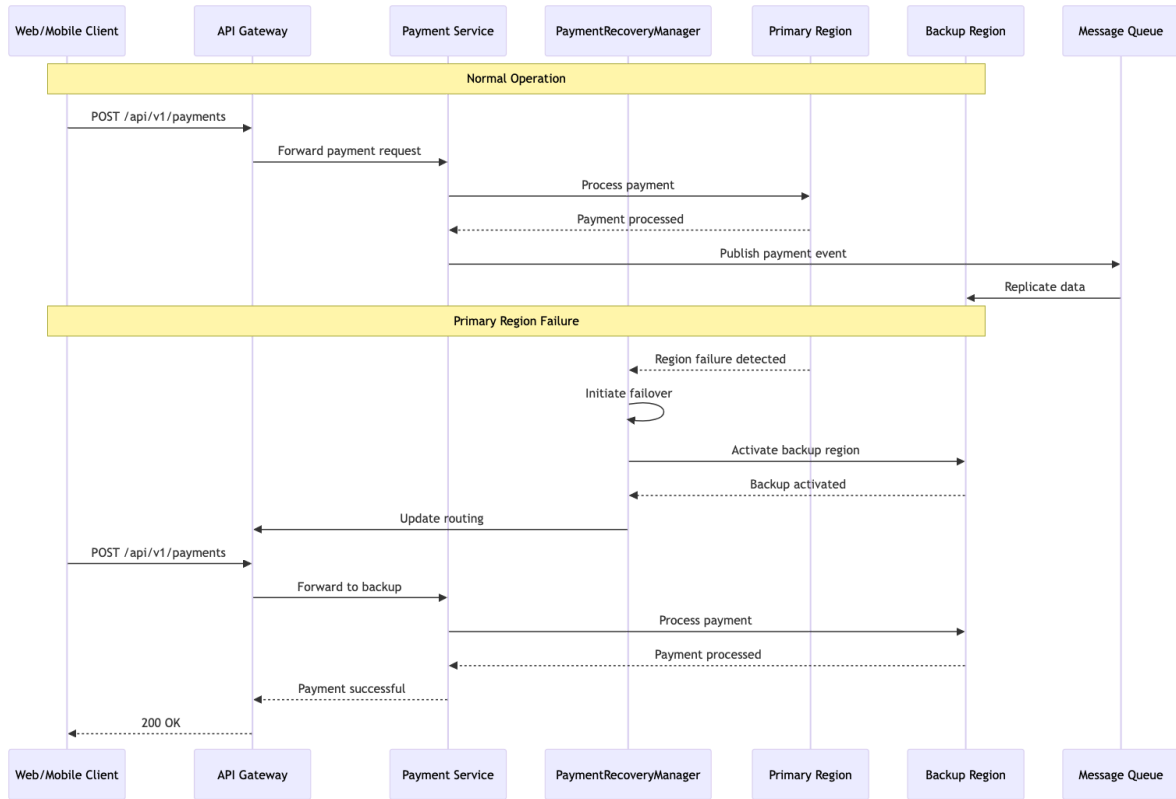


图12 QAS015的序列图

以QAS015系统为例（见图12），支付恢复管理器（PaymentRecoveryManager）会触发故障转移机制、激活备用区域并请求更新路由。虽然该组件在文档其他部分未被提及，但类似组件如工单恢复管理器（Ticket-RecoveryManager）和交付恢复管理器（DeliveryRecoveryManager）的存在，使得设计文档中明确标注了“需采用具备故障转移和备份策略的恢复管理器组件”。这体现了设计层面的矛盾。此外，由于该方案主张由服务内部组件负责恢复管理，而非采用全局性架构——即由各服务共同承担的职责——因此未获采纳。尽管评审人员发现了设计缺陷，但他们对架构文档的完整性表示赞赏，并认为这种方案对实践者极具参考价值。

我们现已能解答研究问题2（RQ2）：实验中，大型语言模型（LLM）生成的架构设计方案虽部分符合架构驱动因素，但架构师并未要求LLM对生成方案进行大量修正。我们认为，若能在方案生成时就进行细致的审查与修正，最终设计将更符合架构驱动因素的要求。

5.3 无 ADD 的设计

为解答第三个研究问题RQ3“当要求语言模型（LLM）设计架构时，使用ADD流程是否会产生显著影响？”，我们让LLM在未提供ADD流程及相关迭代计划的情况下进行设计。为探究ADD流程对设计输出的影响程度，我们开展了三项实验（详见表5）。第一项实验采用零样本方法；第二项实验向LLM提供了空白架构文档模板；第三项实验在模板中添加了补充说明，例如：“针对每个用例和质量属性场景，需包含展示其运作机制的序列图”。

components collaborate to achieve the requirement. Below each sequence diagram, add a short explanation of what is shown in the diagram.” Links to the results of the designs are provided in the result column of the table.

Approach	Prompt	Result
Zero-shot	Consider the requirements described in @ArchitecturalDrivers.md. Design an architecture for this system that satisfies all the requirements described and document this architecture in a document called Architecture.md in the @Design folder. If you include diagrams, use mermaid syntax.	https://github.com/otrebmu/HotelPricingSystem/blob/main/OtherDesigns/ArchitectureZeroShot.md
Template with the headers of the architecture document	Consider the requirements described in @ArchitecturalDrivers.md. Design an architecture for this system that satisfies all the requirements described and document this architecture. Document the design in the @Architecture.md document. If you include diagrams, use mermaid syntax.	https://github.com/otrebmu/HotelPricingSystem/blob/main/OtherDesigns/ArchitectureEmptyTemplate.md
Template with short instructions and empty table.	Consider the requirements described in @ArchitecturalDrivers.md. Design an architecture for this system that satisfies all the requirements described and document this architecture. Document the design in the @Architecture.md document. Carefully read the lines that start with "Instructions:" in the document and follow these instructions. At the end, remove these lines with instructions. If you include diagrams, use mermaid syntax.	https://github.com/otrebmu/HotelPricingSystem/blob/main/OtherDesigns/ArchitectureTemplateWithInstructions.md

Table 5. Experiments without providing ADD to the LLM

The three resulting documents show a design that follows a similar approach of decomposing the system into microservices for the most important entities, where updates are communicated to the entities using a messaging system. Some observations about the solutions are the following:

- Solution 1 (zero shot): A Price Publisher Service sends prices to external systems, but this is not correct as the Property Management System and the Commercial Analysis system must read them through an API. Redis was mentioned but it is not shown in diagrams. This resulted in the least detailed design.
- Solution 2 (empty template): This solution contains a similar mistake to the previous solution.
- Solution 3 (Template with instructions): Integration with external systems is not very clear as there is an "Integration Service" that communicates using REST/Events with the external systems. The external systems should go through the API gateway to access this integration service, but it is accessed directly in the proposed solution.

The most comprehensive document is the third one, which shows that adding some instructions to the template and empty tables makes an important difference. It seems, though, that the LLM did not follow the instructions so closely as it only included sequence diagrams for the two main use cases.

We can now provide an answer to RQ3: It is clear that using ADD in conjunction with the iteration plan and the architecture document template makes a significant difference in the resulting design. The design produced using ADD is much more detailed and also this process gives the opportunity to the architect to make corrections during the execution of ADD.

组件协作以满足要求。在每个序列图下方，添加一个简短的说明，说明图中显示的内容。在表的结果列中提供设计结果的链接。

方法	提示	结果
零示例	考虑 @ArchitecturalDrivers.md 中描述的需求。设计一个系统体系结构，以满足所述的所有需求，并将此体系结构记录在 @Design 文件夹中的 Architecture .md 文件中。如果包含图表，请使用 mermaid 语法。	https://github.com/otrebmu/HotelPricingSystem/blob/main/OtherDesigns/ArchitectureZeroShot.md
架构文档标题模板	考虑 @ArchitecturalDrivers.md 中描述的需求。设计一个满足所有所述需求的系统体系结构，并记录此体系结构。在 @Architecture .md 文档中记录此设计。如果包含图表，请使用 mermaid 语法。	https://github.com/otrebmu/HotelPricingSystem/blob/main/OtherDesigns/ArchitectureEmptyTemplate.md
模板包含简短说明和空白表格。	考虑@ArchitecturalDrivers.md 中描述的需求。设计一个满足所有需求的系统架构，并记录该架构。在@Architecture .md文档中记录设计。仔细阅读文档中以“Instructions: ”开头的行，并遵循这些说明。最后，删除包含说明的行。如果包含图表，请使用mermaid语法。	https://github.com/otrebmu/HotelPricingSystem/blob/main/OtherDesigns/ArchitectureTemplateWithInstructions.md

表5.未向LLM提供ADD的实验

最终生成的三份文档展示了采用相似设计思路的方案：将系统拆分为关键微服务，通过消息系统实现各实体间的更新同步。关于该解决方案的观察结果如下：

- 方案一（零次发送）：价格发布服务直接向外部系统推送价格数据，但该方案存在缺陷——物业管理系统和商业分析系统需通过API接口获取数据。虽然文中提及Redis技术，但未在架构图中体现，最终导致设计方案的细节缺失。
- 方案2（空白模板）：该方案与前一方案存在相似错误。
- 方案3（含说明模板）：外部系统集成存在不明确之处，因为存在一个通过REST/Events协议与外部系统通信的“集成服务”。按方案设计，外部系统需经由API网关访问该集成服务，但当前方案中却直接进行了访问。

第三份文件最为详尽，其内容表明在模板中添加若干说明及空白表格能产生显著影响。然而，LLM似乎并未严格遵循这些说明，仅针对两个主要用例绘制了序列图。

我们现已能解答研究问题3（RQ3）：显然，将ADD与迭代计划及架构文档模板结合使用，会对最终设计产生显著影响。通过ADD生成的设计更为详尽，同时该流程还为架构师提供了在ADD执行过程中进行修正的机会。

6. Threats to validity

One of the most important threats to validity in this study is the possibility of the LLM to have “read” the case study from the [Cervantes2024] book during training. We asked the LLM the following prompt: “Are you familiar with the Hotel Pricing System case study from the book "Designing Software Architectures: A Practical Approach" (2nd edition) by Humberto Cervantes and Rick Kazman. If so, do you have details about this case study?” and the response was “I don't have specific details about the Hotel Pricing System case study from the book "Designing Software Architectures: A Practical Approach" (2nd edition) by Humberto Cervantes and Rick Kazman [...] If you're looking for specific details about this case study, you would need to consult the book directly.” While we cannot be sure the LLM has not been trained to disclose copyright materials that may have been used during training, it seems to be that the case study was not used during the training process. Furthermore, the proposed solution to the hotel pricing system shows a different approach to what is proposed in the book.

Another threat to validity is that the Event Ticketing System is a type of system which lends itself well as an example. The LLM may have also “read” case studies similar to this one during its training. We indeed found some online examples:

- “Design a Ticket Booking Site Like Ticketmaster”⁵.
- “A case study on online ticket booking system”⁶
- “Online events ticketing management system, a case study of Namboole stadium”⁷
- “Scalable Ticketing Platform Development for Good Ticket Company - U.S. Entertainment and Ticketing Brand”⁸

Of these examples, only the Ticketmaster case study goes into deeper technical detail about the architecture. This solution addresses a very similar challenge but many design decisions differ, as shown in Appendix B. These differences allow us to conclude that the solution proposed by the LLM is significantly different. A final threat to validity for this study is that the evaluation with professional architects was made with a very small sample (2 people) from the same company. Also the scenario-based evaluation meeting was constrained in time, so the depth of analysis was limited.

7. Analysis and discussion

The two case studies that were used for validation show encouraging results. The architecture documents that were produced in both cases are of high quality and more comprehensive than many examples that we have observed in real projects in industry, and they were produced in a very short time (performing the actual design activity only required a couple of hours each). One key contribution of our approach is the idea of providing an explicit design process to guide the LLM in the design activity. Another important idea is that of asking the LLM to plan design

⁵ <https://www.hellointerview.com/learn/system-design/problem-breakdowns/ticketmaster>

⁶ <https://advance.sagepub.com/users/454994/articles/1212909/master/file/data/A%20CASE%20STUDY%20ON%20ONLINE%20TICKET%20BOOKING%20SYSTEM%20PROJECT/A%20CASE%20STUDY%20ON%20ONLINE%20TICKET%20BOOKING%20SYSTEM%20PROJECT.pdf>

⁷

http://www.researchgate.net/publication/375962782_ONLINE_EVENTS_TICKETING_MANAGEMENT_SYSTEM_A_CASE_STUDY_OF_NAMBOOLE_STADIUM

⁸ <https://pagepro.co/case-studies/good-ticket-company>

6. 效度威胁

本研究中对有效性构成最大威胁的因素之一，是大型语言模型（LLM）在训练过程中可能“阅读”过《塞万提斯2024》书籍中的案例研究。我们向LLM提出了以下提问：“您是否熟悉洪贝托·塞万提斯与里克·卡兹曼合著的《软件架构设计：实用方法》（第二版）中关于酒店定价系统的案例研究？如果有，您是否掌握该案例的具体细节？”其回答为：“关于洪贝托·塞万提斯与里克·卡兹曼合著的《软件架构设计：实用方法》（第二版）中酒店定价系统的案例研究，我并无具体细节[...]若您需要获取该案例的具体细节，需直接查阅原著。”虽然我们无法确定LLM是否在训练过程中被训练泄露可能使用的版权材料，但可以确定的是，该案例研究并未在训练过程中被使用。此外，针对酒店定价系统提出的解决方案，其方法论与书中所述存在显著差异。

效度的另一威胁在于，事件票务系统（Event Ticketing System）本身便是一个典型的示例系统。法学硕士（LLM）在培训期间可能也“阅读”过与此类案例相似的研究。我们确实发现了一些在线示例：

- “设计一个像Ticketmaster一样的票务网站”⁵。
- “在线票务系统”案例研究⁶
- “在线活动票务管理系统——以南布勒体育场为例”⁷
- “面向优质票务公司的可扩展票务平台开发——美国娱乐票务品牌”⁸

在这些案例中，唯有票务大师（Ticketmaster）的案例研究对系统架构进行了更深入的技术解析。虽然该方案应对了类似挑战，但正如附录B所示，其设计决策存在诸多差异。这些差异使我们得出结论：该LLM提出的解决方案存在显著不同。本研究效度的最后威胁在于，专业架构师的评估仅基于同一家公司极小样本（2人）。此外，基于场景的评估会议时间受限，导致分析深度不足。

7. 分析和讨论

用于验证的两个案例研究取得了令人鼓舞的成果。在这两个案例中生成的架构文档不仅质量上乘，其全面性甚至超越了我们在工业实际项目中观察到的许多范例，而且完成时间极为紧迫（实际设计工作仅需数小时即可完成）。我们方法的核心贡献在于：通过提供明确的设计流程来指导语言模型开展设计活动。另一个重要创新点是让语言模型主动规划设计方案。

<https://www.hellointerview.com/learn/system-design/problem-breakdowns/ticketmaster>

⁶<https://advance.sagepub.com/users/454994/articles/1212909/master/file/data/A%20CASE%20STUDY%20ON%20ONLINE%20TICKET%20BOOKING%20SYSTEM%20PROJECT/A%20CASE%20STUDY%20ON%20ONLINE%20TICKET%20BOOKING%20SYSTEM%20PROJECT.pdf>

⁷

http://www.researchgate.net/publication/375962782_ONLINE_EVENTS_TICKETING_MANAGEMENT_SYSTE 南博莱体育场案例研究

⁸ <https://pagepro.co/case-studies/good-ticket-company>

work before performing the actual design through the iteration plan. This is very useful both for the LLM and for the user to know when to stop designing. Without this plan, there is a risk of the LLM producing an excessively complex Big Design Up Front (BDUF). Another fundamental aspect of our approach is the use of a unique architecture document that is modified throughout the design process. In our initial experiments, we documented design using only the iteration documents (following an approach similar to the Hotel Pricing System case study in the book), but this quickly led to problems as each iteration was resulting in structures focused on the drivers of the iteration but there was not an overarching design resulting from these iterations. One limitation of this approach, however, is that as the architecture document grows large, the LLM starts to struggle when making changes to it and this sometimes results in mistakes.

One important caveat to consider is that the two solutions used for validation were created using an approach where the user did not take a considerable time reviewing and correcting the LLM about its decisions at the end of each ADD step. This was made on purpose because we wanted to study how good the design could be performed “out of the box”. However, there were a number of problems that occurred during the design process and sometimes light corrections were suggested to the LLM. Examples of design problems encountered in the design of the event ticket system are the following.

- When creating the domain model, it did not consider the cloud provider identity service. This resulted in a User entity more complex than needed. The LLM was reminded of the use of the identity service and corrections were made.
- The venue management system, which appears in the context diagram, is not present in the container diagram and is largely ignored in the solution.
- After decomposing several components using a layered approach, the LLM decided to use a hexagonal architecture in the 6th iteration. We mentioned this problem and the LLM accepted the mistake and proceeded to correct it.
- The LLM wanted to introduce a service mesh later in the design. When questioned about the complexity of this approach, it accepted that the same outcome could be obtained by the use of an enhanced api gateway and it proceeded to change the design.

In addition to these design problems, we also faced constant issues related to the LLM “forgetting” information, for example

- It did not present the information as described in step 4 of Attribute Driven Design (see figure 3).
- Frequently forgot to stop after each step and wait for review.
- Forgot to modify the architecture file and started to work on the iteration files exclusively.
- Forgot to adjust the container diagrams.
- Forgot to add design decisions to the architecture document.
- Added sequence diagrams for drivers that were not being addressed as part of the iteration.

Our belief is that many of these problems are related to the amount of context that the LLM receives as part of each prompt. This context is provided by the IDE tool and is beyond our control. In the Event Ticketing System, we also decided to have each user story documented in a separate file and this posed challenges for passing it to the LLM as part of the context. Even though the IDE allows an entire directory to be passed as part of the context, the design decisions show that the user story documents may not have been read by the LLM as expected.

Considering the results we have obtained so far, we believe that it is not yet possible to perform a “*vibe* architecting” approach where we only provide an LLM with requirement information and ask it to produce the design. Architectural design is a complex activity, and performing it automatically is still outside the possibilities of current LLMs (fortunately!). However, the approach we present can be very beneficial if architecture design is performed as a collaboration between the architect and the LLM. The design process can be accelerated significantly, and

在实际设计之前通过迭代计划进行工作。这对于大型语言模型和用户来说都非常有用，可以知道何时停止设计。没有这个计划，大型语言模型可能会产生过于复杂的前期大设计（BDUF）。我们方法的另一个基本方面是使用一个独特的架构文档，该文档在整个设计过程中不断修改。在最初的实验中，我们仅使用迭代文档记录设计（采用类似于书中酒店定价系统案例研究的方法），但这很快导致了问题，因为每次迭代都产生了以迭代驱动因素为中心的结构，但这些迭代并没有形成一个总体设计。然而，这种方法的一个局限性是，随着架构文档变得庞大，大型语言模型在对其进行修改时开始遇到困难，有时会导致错误。

需要特别注意的是，用于验证的两个解决方案采用了特殊设计：在每次ADD步骤结束时，用户并未花费大量时间审查和修正大语言模型（LLM）的决策。这种设计是有意为之，因为我们想研究该设计方案在“开箱即用”状态下能实现多高的优化效果。不过在设计过程中确实出现了一些问题，有时需要对LLM进行轻微调整。以下是事件票务系统设计过程中遇到的典型问题示例。

- 在创建领域模型时，未考虑云服务提供商的身份服务，导致用户实体过于复杂。LLM被提醒需使用身份服务，并进行了修正。
- 场景管理系统的存在（如上下文图所示）在容器图中未体现，且在解决方案中基本被忽略。
- 在采用分层方法分解多个组件后，LLM于第六次迭代中决定采用六边形架构。我们已提及该问题，LLM承认错误并着手进行修正。
- LLM计划在后续设计阶段引入服务网格。当被问及该方案的复杂性时，其承认采用增强型API网关可实现相同效果，并据此调整了设计方案。

除这些设计问题外，我们还持续面临与大语言模型（LLM）‘遗忘’信息相关的挑战，例如

- 该信息未按照属性驱动设计（Attribute Driven Design）第4步所述方式呈现（参见图3）。
- 经常忘记在每一步骤后停止并等待审查。
- 忘记修改架构文件，转而专门处理迭代文件。
- 忘记调整容器示意图。
- 未将设计决策添加至架构文档。
- 新增了迭代阶段未涵盖的驱动程序序列图。

我们发现，这些问题大多与大型语言模型（LLM）在每次提示中接收的上下文量有关。这些上下文由集成开发环境（IDE）工具提供，且不受我们控制。在事件工单系统中，我们还决定将每个用户故事单独存档，这给将其作为上下文传递给LLM带来了挑战。虽然IDE支持将整个目录作为上下文传递，但设计决策表明，LLM可能并未如预期般读取用户故事文档。

根据目前的研究成果，我们认为现阶段还无法实现“感知式架构设计”——即仅通过向大语言模型输入需求信息就要求其自动生成设计方案。架构设计本质上是一项复杂的工作，而完全自动化处理仍超出当前大语言模型的能力范围（值得庆幸的是！）。不过，若将架构设计作为建筑师与大语言模型的协作过程，我们提出的方法将大有裨益。这种设计流程不仅能显著提升效率，

excellent documentation is produced as part of the design process. One of the difficulties of this approach, however, is that the user that designs in collaboration with the LLM must have enough experience to be able to review and question the decisions made by the LLM. For less experienced architects, there is a risk of blindly accepting the solution “out of the box” and this may result in the introduction of risks or technical debt to the system. Also, LLMs are still prone to hallucination, so reviewing the work they are generating is still a necessary task.

8. Conclusions and future work

In this paper we have presented an LLM-assisted approach to assist architects in designing software architectures using the Attribute Driven Design process. Our proposal introduces key aspects such as the use of an iteration plan, an explicit architecture design process (ADD), an architect persona and a single architecture design document which is refined over several ADD iterations. The results we have obtained so far are very encouraging, especially considering that there are few available examples of documented architecture designs. Examples such as the ones in the book [Cervantes2024] were, it appears, not used during the training process as they are copyrighted material. This means that the LLM was capable of making reasonable design decisions and creating and documenting the structures that compose an architecture.

The results in this study give us confidence that it is indeed possible to perform LLM-assisted architecture design, but the process has to be done carefully and in close collaboration between the human and the LLM. It is necessary to review the products of each step of the design process and to correct and guide the LLM. If we omit this crucial step of review and correction of the outcomes of the design process we will get designs that are created with inconsistencies or technical debt from the start.

This paper has presented an initial approach to performing LLM-assisted architecture, and there are many future areas of research:

- **Improving context:** We need to study approaches that improve the information that is passed to the LLM during the design process to avoid omissions and memory losses. This may be challenging when using an IDE as we may not have exact control over what is passed to the LLM and outside the IDE, it may also be challenging as there are associated costs to passing input tokens so there is an incentive to limit context as much as possible to reduce costs.
- **Providing additional design knowledge to the LLM:** Even though LLMs are trained on vast amounts of information, they may not have had access to specialized literature related to designing software architectures, such as the Tactics catalog in [Bass2021]. Providing additional design knowledge may improve the outcomes of the design process. IDEs such as Cursor allow additional documentation to be used by the LLM but it has to be attached as part of a specific prompt. An interesting idea would be to use a specialized architect LLM which is already fine tuned with considerable amounts of architectural and design knowledge.
- **Improve design decisions by making tradeoff analysis:** Evaluating tradeoffs when making design decisions is essential. Currently, we are not explicitly asking the LLM to analyze tradeoffs when making its design decisions, and studying how well the LLM can make this type of analysis remains to be done.
- **Evaluating architectures using LLMs:** An interesting idea is to use a different LLM to evaluate the output of the design process. The architecture document facilitates this task enormously and we have already made some informal experiments with encouraging results, but a more formal approach still needs to be investigated. A sample analysis is provided in the the Analysis folder of both projects presented in

在设计过程中会生成详尽的文档资料。但这种方法存在一个难点：与LLM（语言模型）协同设计的用户必须具备足够经验，才能对LLM的决策进行审阅和质疑。对于经验不足的架构师而言，存在盲目接受“开箱即用”解决方案的风险，这可能导致系统引入风险或技术债务。此外，LLM仍存在认知偏差的倾向，因此对其生成的工作成果进行复核仍是必要环节。

8. 结论与未来工作

本文提出了一种基于大语言模型（LLM）的辅助方法，旨在帮助架构师运用属性驱动设计（ADD）流程进行软件架构设计。我们的方案引入了迭代计划、显性架构设计流程（ADD）、架构师角色模型以及单一架构设计文档等关键要素，这些要素通过多次ADD迭代不断优化完善。目前取得的成果令人鼓舞，尤其考虑到现有文档化架构设计案例凤毛麟角。例如《塞万提斯2024》一书中的案例显然未被纳入训练过程，因其属于受版权保护的材料。这表明大语言模型能够做出合理的设计决策，并能创建和记录构成架构的各个组件结构。

本研究结果让我们确信，利用大语言模型辅助架构设计确实可行，但必须谨慎操作并保持人机深度协作。需要对设计流程的每个阶段成果进行复盘，及时修正并引导模型优化。若省略这个关键的复盘修正环节，最终设计成果将从一开始就存在逻辑矛盾或技术债务问题。

本文提出了一种实现LLM辅助架构的初步方法，未来研究领域广阔：

- **提升上下文质量：**我们需要研究优化设计流程中传递给大语言模型（LLM）的信息方法，以避免信息遗漏和记忆缺失。在使用集成开发环境（IDE）时，这可能面临挑战——我们无法完全控制传递给LLM的具体内容。此外，由于传递输入标记会产生相关成本，开发者往往倾向于尽可能限制上下文信息，从而降低处理成本。
- **为语言模型补充设计知识：**尽管语言模型经过海量数据训练，但可能缺乏软件架构设计领域的专业文献资源，例如[Bass2021]中提到的Tactics设计目录。补充相关设计知识能有效提升设计流程的成效。像Cursor这类集成开发环境虽支持语言模型调用补充文档，但必须将其作为特定提示的组成部分。一个创新思路是采用经过大量架构设计知识微调的专业架构语言模型，这类模型已具备丰富的设计经验。
- **通过权衡分析优化设计决策：**在设计决策过程中评估利弊得失至关重要。目前我们尚未明确要求大语言模型在设计决策时进行权衡分析，关于大语言模型能否胜任此类分析的研究仍有待开展。
- **利用大语言模型评估架构设计：**一个颇具创意的思路是采用不同的大语言模型来评估设计流程的输出结果。架构文档为这项任务提供了极大便利，我们已通过非正式实验取得了积极成果，但更系统化的评估方法仍有待深入研究。具体分析案例可参见两个项目中“Analysis”文件夹的展示。

this paper⁹ and ¹⁰. An interesting possibility is to perform an automated evaluation at the end of every iteration and provide the feedback from the evaluating LLM to the designing LLM.

- **Generating code from the design:** The next logical step from the design process is to generate the actual system code by giving the LLM the architecture document. Similar to the evaluation activity, we have started experimenting this approach with the HPS design. We have observed discrepancies between code and design (probably due again to a problem with prompt context) so we still need to study how to improve this approach with more detail.
- **Continuous changes to the design:** The design approach presented here is not a Big Design Up Front approach. Ideally, a few ADD iterations are performed at the beginning of a project in a design round and additional iterations are performed later in other iteration rounds, as the system is being built. We need to explore the process of performing continuous design iterations rounds and how these impact the documentation and the code.
- **Updating the architecture document from changes in the code:** Ideally the architecture document should always reflect the state of the code, however, it may happen that changes are done to the code without the architecture documentation being updated. This can be problematic if we intend to perform additional ADD iterations on an outdated architectural model. Automatically updating the architectural document from changes in the code is an interesting research area.
- **Use of other LLMs:** the experiments conducted in this paper were done using the same LLM: Claude Sonnet 3.7. It may be that other LLMs may produce better results so a comparison remains to be done.

Finally, it is important to consider whether or not it is desirable to reach a point where LLMs can design software architectures automatically. Our opinion, at this time, is that design is a creative act that is better performed as a collaboration between a human and an LLM.

9. References

[Bachmann2003] Bachmann, F., Bass, L., & Klein, M. (2003). “Preliminary Design of ArchE: A Software Architecture Design Assistant” (CMU/SEI-2003-TR-021, ESC-TR-2003-021). Software Engineering Institute, Carnegie Mellon University.

[Bass2021] Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice* (4th ed.). Addison-Wesley Professional. ISBN: 978-0-13-688609-9.

[Brown2018] Brown, S. (2018). “The C4 model for visualising software architecture.” <https://c4model.com/>

[Bucaioni2025] Bucaioni, A., Weyssow, M., He, J., Lyu, Y., & Lo, D. (2025). “Artificial Intelligence for Software Architecture: Literature Review and the Road Ahead”. arXiv preprint arXiv:2504.04334.

[Cervantes2024] Cervantes, H., & Kazman, R. (2024). “Designing Software Architectures: A Practical Approach (2nd ed.)”. Addison-Wesley Professional. ISBN: 978-0-13-810802-1.

[Dhar2025] Dhar, R., Kakran, A., Karan, A., Vaidhyanathan, K., & Varma, V. (2025). “DRAFT-ing Architectural Design Decisions using LLMs”. arXiv preprint arXiv:2504.08207.

⁹ <https://github.com/otrebmu/HotelPricingSystem/blob/main/Analysis/AnalysisResults.md>

¹⁰ <https://github.com/otrebmu/EventTicketSystem/blob/master/Analysis/AnalysisResults.md>

本文⁹和¹⁰。一个有趣的可能性是在每次迭代结束时进行自动化评估，并将评估LLM的反馈提供给设计LLM。

- **从设计生成代码：**设计流程的下一步自然就是将架构文档输入到大语言模型（LLM）中，生成实际的系统代码。与评估活动类似，我们已开始在 HPS 设计中测试这种方法。不过我们发现代码与设计之间存在差异（可能再次归因于提示上下文的问题），因此仍需深入研究如何改进这一方法。
- **持续性设计迭代：**本文提出的设计方法并非采用“前期大设计”模式。理想情况下，项目初期会在设计轮次中完成几次ADD迭代，而在系统开发过程中，后续迭代轮次会继续进行更多迭代。我们需要深入探讨持续设计迭代轮次的实施流程，以及这些迭代对文档编写和代码实现产生的影响。
- **代码变更与架构文档的同步机制：**理想情况下，架构文档应当实时反映代码状态。但现实中，代码更新时往往忽略同步架构文档，这可能导致后续在过时架构模型上进行迭代开发时出现严重问题。如何实现代码变更与架构文档的自动同步，已成为当前值得深入研究的前沿课题。
- **其他LLM的使用：**本文中进行的实验使用的是相同的LLM：Claude Sonnet 3.7。其他LLM可能产生更好的结果，因此仍需进行比较。

最后，需要考虑的是，让大型语言模型（LLM）实现软件架构的自动化设计是否具有实际意义。我们认为，设计本质上是一种创造性活动，更适宜通过人机协作来完成。

9. 参考文献

[Bachmann2003] 巴赫曼（F.）、巴斯（L.）与克莱因（M.）于2003年联合发表《ArchE：软件架构设计辅助工具的初步设计》（项目编号：CMU/SEI-2003-TR-021，ESC-TR-2003-021），该成果由卡内基梅隆大学软件工程研究所发布。

[Bass2021] Bass, L., Clements, P., & Kazman, R. (2021). 《软件架构实践》（第4版）。艾迪生-韦斯利专业出版社。ISBN: 978-0-13-688609-9。

[Brown2018] Brown, S. (2018). “用于可视化软件架构的C4模型。” <https://c4model.com/>

[Bucaioni2025] Bucaioni, A., Weyssow, M., He, J., Lyu, Y., & Lo, D. (2025). “人工智能在软件架构中的应用：文献综述与未来方向”。arXiv预印本 arXiv: 2504.04334。

[塞万提斯2024] 塞万提斯, H., 与卡兹曼, R. (2024) 合著。《软件架构设计：实用方法（第二版）》。艾迪生-韦斯利专业出版社。ISBN: 978-0-13-810802-1。

[Dhar2025] Dhar, R., Kakran, A., Karan, A., Vaidhyathan, K., & Varma, V. (2025). “使用LLMs进行建筑设计决策的草稿”。arXiv预印本 arXiv: 2504.08207。

⁹ <https://github.com/otrebmu/HotelPricingSystem/blob/main/Analysis/AnalysisResults.md>

¹⁰ <https://github.com/otrebmu/EventTicketSystem/blob/master/Analysis/AnalysisResults.md>

- [Eisenreich2024] Eisenreich, T., Speth, S., & Wagner, S. (2024). “From Requirements to Architecture: An AI-Based Journey to Semi-Automatically Generate Software Architectures.” arXiv preprint arXiv:2401.14079.
- [Esposito2025] Esposito, M., Li, X., Moreschini, S., Ahmad, N., Cerny, T., Vaidhyanathan, K., Lenarduzzi, V., & Taibi, D. (2025). “Generative AI for Software Architecture: Applications, Trends, Challenges, and Future Directions”. arXiv preprint arXiv:2503.13310.
- [Evans2003] Evans, E. (2003). “Domain-Driven Design: Tackling Complexity in the Heart of Software”. Addison-Wesley Professional.
- [Helmi2025] Helmi, T. (2025). “ARLO: A Tailorable Approach for Transforming Natural Language Software Requirements into Architecture using LLMs”. arXiv preprint arXiv:2504.06143.
- [Jahić2024] Jahić, J., & Sami, A. (2024). State of Practice: LLMs in Software Engineering and Software Architecture. In Proceedings of the 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C) (pp. 311–318). IEEE. <https://doi.org/10.1109/ICSA-C63560.2024.00059>
- [Maranhão2024] Maranhão Junior, J. J., Melegati, J., & Guerra, E. (2024). A Prompt Pattern Sequence Approach to Apply Generative AI in Assisting Software Architecture Decision-making. In Proceedings of the 29th European Conference on Pattern Languages of Programs (EuroPLOP '24). Association for Computing Machinery. <https://doi.org/10.1145/3698322.3698324>
- [Palma2012] F. Palma, H. Farzin, Y.-G. Guéhéneuc, and N. Moha, “Recommendation system for design patterns in software development: An DPR overview,” in 2012 Third International Workshop on Recommendation Systems for Software Engineering (RSSE). IEEE, 2012, pp. 1–5.
- [Rivera2024] Rivera Hernández, B. M., Santos Ayala, J. M., & Méndez Melo, J. A. (2024). “Generative AI for Software Architecture” (Undergraduate thesis, Universidad de los Andes). <https://hdl.handle.net/1992/74979>
- [Supekar2024] Supekar, V., & Khande, R. (2024). Improving Software Engineering Practices: AI-Driven Adoption of Design Patterns. In Proceedings of the 2024 IEEE International Conference on Advanced Computing Technologies (ICACCTech), pp. 768–774. IEEE.
- [Wang2023] Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K.-W., & Lim, E.-P. (2023). “Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models”. arXiv preprint arXiv:2305.04091.
- [Wei2024] Wei, B. (2024). Requirements are All You Need: From Requirements to Code with LLMs. arXiv preprint arXiv:2406.10101.

[Eisenreich2024] Eisenreich, T., Speth, S., & Wagner, S. (2024). “从需求到架构：基于人工智能的半自动生成软件架构之旅。” arXiv预印本 arXiv: 2401.14079。

[Esposito2025] Esposito, M., Li, X., Moreschini, S., Ahmad, N., Cerny, T., Vaidhyathan, K., Lenarduzzi, V., & Taibi, D. (2025). “软件架构的生成式人工智能：应用、趋势、挑战及未来方向”。arXiv预印本 arXiv: 2503.13310。

[Evans2003] Evans, E. (2003). 《领域驱动设计：攻克软件核心复杂性》
艾迪生-韦斯利专业公司。

[Helmi2025] Helmi, T. (2025). “ARLO：一种可定制的方法，利用大型语言模型将自然语言软件需求转化为架构”。arXiv 预印本 arXiv: 2504.06143。

[Jahic2024] Jahic, J., & Sami, A. (2024)。实践现状：软件工程与软件架构中的LLM。收录于《2024年IEEE第21届国际软件架构会议论文集（ICSA -C）》（第311-318页）。IEEE。https://doi.org/10.1109/ICSA-C63560.2024.00059

[Maranhao2024] Maranhao Junior, J. J., Melegati, J., & Guerra, E. (2024). 基于提示模式序列方法将生成式AI应用于辅助软件架构决策。第29届欧洲程序模式语言会议论文集（EuroPLOP '24）。美国计算机协会。https://doi.org/10.1145/3698322.3698324

[Palma2012] F. Palma, H. Farzin, Y.-G. Gueheneuc 和 N. Moha, “软件开发设计模式推荐系统：DPR概述”，在 2012 年第三届国际软件工程推荐系统研讨会（RSSE）上。IEEE, 2012, 第 1-5 页。

[Rivera2024] Rivera Hernandez, B. M., Santos Ayala, J. M., & Mendez Melo, J. A. (2024). “软件架构的生成式AI”（本科论文，安第斯大学）。https://hdl.handle.net/1992/74979

[Supekar2024] 苏佩卡尔, V., 与坎德, R. (2024)。改进软件工程实践：基于人工智能的设计模式采用。收录于《2024年IEEE国际先进计算技术会议论文集》（ICACCTech），第768–774页。IEEE。

[Wang2023] 王立、徐伟、兰燕、胡振、李瑞宽和林永鹏 (2023)。 “计划-求解提示：通过大型语言模型提升零样本链式推理能力”。arXiv预印本 arXiv:2305.04091.

[Wei2024] Wei, B. (2024). 需求就是一切：从需求到代码的LLM指南。arXiv 预印本 arXiv: 2406.10101。

Appendix A

The following table presents a comparison between the solution produced by the LLM and the one that is documented in the case study of [Cervantes2024].

Aspect	Case study	LLM generated solution
System Structure	The design splits the system into a Command service (for writes) and Query service (for reads) following Command Query Responsibility Segregation (CQRS), plus an Export service for integration. These services are deployed behind a single API Gateway in a private cloud network. The Command and Query components communicate via an event bus (Kafka) instead of direct calls. This results in only a few core back-end services (command, query, export) that handle all pricing functionality.	The design uses a microservices approach but with more fine-grained separation. In addition to Price Management (command) and Price Query (read) services, there are distinct services for Hotel Management, Rate Management, and Authentication. An API Gateway fronts all these services. An Event Bus enables asynchronous communication between services. The existence of the Query service is essentially an implementation of CQRS.
Components	Each microservice in the design is structured using standard Spring layered architecture. This mapping of entities to controllers/services ensures all required modules are identified for each use case. The focus is on fundamental components needed for functionality, with security and basic logging considered.	Each service in the design contains multiple specialized components additional to the standard Spring Layers components (e.g. Calculation Engine, Event Store, Event Publisher with Outbox, Circuit Breaker). This design is more complex and incorporates patterns for reliability and performance.
Sequence diagrams	The case study presents a limited number of sequence diagrams to illustrate certain functional and non functional scenarios	The architecture document includes sequence diagrams for all functional and quality attribute scenarios.
Contracts	In iteration 2, a few container and component-level contracts are presented, mostly for illustration purposes.	The architecture document includes detailed container-level REST contracts for the different services. Endpoints are described in tables with methods, descriptions, request body and response. Additionally, description of events and event payloads is also included.

附录A

下表展示了由LLM生成的解决方案与[Cervantes2024]案例研究中记录的解决方案之间的对比。

方面	个案研究	LLM生成的解决方案
系统结构	设计将系统分为命令服务（用于写入）和查询服务（用于读取），遵循命令查询责任分离（CQRS）原则，并增加了一个导出服务用于集成。这些服务部署在私有云网络中的单一API网关后。命令和查询组件通过事件总线（Kafka）通信，而不是直接调用。这使得只有少数核心后端服务（命令、查询、导出）处理所有定价功能。	该设计采用微服务架构，但实现了更细粒度的分离。除价格管理（命令）和价格查询（读取）服务外，还分别设有酒店管理、费率管理和身份验证等独立服务。所有服务均通过API网关进行统一调用，事件总线则实现了服务间的异步通信。其中，查询服务的存在本质上是CQRS模式的实现。
组件	该设计中的每个微服务均采用标准Spring分层架构进行构建。通过将实体映射至控制器/服务，确保每个用例都能识别所有必需模块。重点在于功能实现所需的核心组件，同时兼顾安全性和基础日志记录。	该设计中的每个服务除标准Spring Layers组件外，还包含多个专业组件（如计算引擎、事件存储、带Outbox的事件发布者、断路器）。该设计更为复杂，采用了可靠性与性能优化的模式。
序列图	该案例研究仅提供了少量序列图，用以说明某些功能性和非功能性场景。	该架构文档包含所有功能与质量属性场景的序列图。
合同	在第二轮迭代中，提出了若干容器级和组件级的合同，主要出于说明目的。	该架构文档包含针对不同服务的详细容器级REST接口协议。端点通过表格形式描述，包含方法、描述、请求体及响应信息。 此外，还包括事件描述及事件负载的说明。

Communication Style	The design emphasizes asynchronous communication. The Command service publishes price change events to an event bus, which the Query and Export services subscribe to. External interactions use synchronous REST calls via the API Gateway, but internal services exchange data through events.	The design employs an event-driven architecture for inter-service communication. Services publish events (hotel updates, rate changes, price changes) to a centralized Event Bus and subscribe where needed. The architecture also provides synchronous APIs through the API Gateway for clients. The Query service supports multiple protocols: both REST and gRPC APIs are offered for retrieving prices.
Domain Modeling	In iteration 2, an explicit domain model is defined for the Hotel Pricing context. Key domain entities include Hotel, RoomType, Rate, and Price, along with business rules for price calculations. This domain model is confined to the command side (write model). The Query service does not maintain an object model of its own; it simply stores price events in a database for fast retrieval, and the Export service likewise has no domain data (just passes events along). Thus, the case study design treats “hotel pricing” as one domain context, centrally modeled in the command microservice.	The design models similar core concepts (Hotel, Rate, RoomType, Price, BusinessRule, etc.), but splits responsibility across services. This design follows a Domain-Driven Design approach by dividing the domain model across context-specific services.
API Management	The case study employs an API Gateway as the single entry point for all clients (users or external systems). The Gateway forwards requests to the appropriate back-end microservice. Each microservice exposes RESTful endpoints for its functionality. Authentication and basic authorization are enforced at the gateway and service level (integrating with the cloud identity service). There is no mention of alternate protocols or versioning strategies in the early design; the focus is on providing the necessary REST endpoints for each use case. API security (e.g. token validation) and request validation tactics are considered as part of securing these APIs.	The design also uses a central API Gateway that routes requests to the appropriate service and applies cross-cutting concerns like auth, rate limiting, and input validation. Each microservice defines a set of APIs (documented in the architecture’s Interface section) for its domain. The design explicitly supports API versioning and multiple protocols. In particular, the Price Query service offers both a REST API and a gRPC interface for clients with different needs.

通信方式	该设计以异步通信为核心。命令服务将价格变更事件发布至事件总线，查询服务与导出服务均订阅该总线。外部交互通过API网关使用同步REST调用，而内部服务则通过事件机制进行数据交换。	该系统采用事件驱动架构实现服务间通信。各服务通过事件总线发布更新（如酒店动态、房价调整、价格变动）并按需订阅。系统还通过API网关为客户端提供同步API接口。查询服务支持多种协议，提供REST和gRPC两种API接口用于获取价格信息。
领域模型	在第二轮迭代中，我们为酒店定价场景构建了明确的领域模型。核心领域实体包括酒店、房型、房价和价格，以及价格计算的业务规则。该领域模型仅限于命令端（写入模型）使用。查询服务没有维护自己的对象模型，仅将价格事件存储在数据库中以便快速检索；导出服务同样不包含领域数据（仅负责事件传递）。因此，本案例研究将“酒店定价”视为一个统一的领域上下文，并在命令微服务中进行集中建模。	该设计模型采用相似的核心概念（如酒店、房型、价格、业务规则等），但将职责分散到不同服务中。这种设计遵循领域驱动设计方法，通过将领域模型划分为特定情境的服务来实现。
API管理	本案例研究采用API网关作为所有客户端（用户或外部系统）的统一入口。该网关将请求转发至对应的后端微服务，每个微服务都通过RESTful接口提供功能支持。身份验证与基础授权机制在网关和服务层面实施（与云身份服务集成）。早期设计中未涉及替代协议或版本控制策略，重点在于为各类应用场景提供必要的REST接口。 API安全措施（如令牌验证）和请求验证策略被纳入API防护体系。	该设计采用中央API网关架构，通过智能路由将请求分配至对应服务，并统一处理身份验证、速率控制和输入验证等跨领域通用功能。每个微服务都会为其业务领域定义专属API集（具体规范详见架构文档的接口章节）。系统明确支持API版本控制和多协议兼容，其中价格查询服务特别为不同需求的客户端同时提供REST API和gRPC接口。

Security	User login (HPS-1) is handled by leveraging the company's cloud-based User Identity Service for Single Sign-On. When a user provides credentials, HPS delegates verification to this identity service and, upon success, grants access with appropriate permissions. Each API call must be authenticated (e.g., via tokens issued by that identity service). Authorization checks are mentioned as a requirement, but how this is handled is not discussed. In iteration 1, security tactics such as authenticating and authorizing actors, limiting access to resources, and encrypting API data were identified as critical and were planned from the start.	The design introduces an Authentication Service as a dedicated microservice to handle auth concerns. The API Gateway delegates authentication requests to this Auth service. The Auth service in turn integrates with the same cloud identity provider via OAuth2/OIDC protocols to validate credentials and tokens. It likely manages user sessions or JWT tokens and also stores authorization data. The design explicitly mentions using the cloud identity (CON-2) for user management and SSO, similar to the case study approach, but here it's encapsulated in a service.
Scalability	A primary motivation for the case study's CQRS and microservice design was to address scalability and performance. By splitting the query side from the command side, the system can scale reads independently of writes. In iteration 3, the design explicitly adds replication for the Query service and load balancing to increase throughput for price queries. Kafka as an event store also allows scaling consumers (multiple query or export processors) if needed. This approach ensures that heavy read load from external systems querying prices can be met by adding resources to the Query microservice alone. Other scalability considerations include avoiding shared state (each microservice has its own database) and using stateless services to facilitate horizontal scaling.	The architecture considers independent scaling of each microservice. For instance, if price queries spike, the Price Query Service can be scaled to many instances without touching the others. The inclusion of a distributed caching layer in the Query service supports high read throughput by reducing database load. Additionally, the architecture accounts for scaling in multiple dimensions: it uses CQRS to divide read/write load, and it can leverage multi-region deployment to distribute load and reduce latency for users in those regions. The decision to containerize everything and orchestrate via Kubernetes means the platform can auto-scale services based on demand.

保护措施	用户登录（HPS -1）通过公司基于云的单点登录用户身份服务来处理。当用户提供凭证时，HPS 将验证委托给该身份服务，成功后授予相应的权限访问。每次 API 调用都必须进行身份验证（例如，通过该身份服务颁发的令牌）。虽然提到了授权检查的要求，但并未讨论如何处理。在第一轮迭代中，识别出诸如验证和授权操作者、限制资源访问以及加密 API 数据等安全策略至关重要，并且从一开始就进行了规划。	设计中引入了一个认证服务作为专用微服务来处理认证问题。API 网关将认证请求委托给该认证服务。认证服务通过 OAuth2/ OIDC 协议与同一云身份提供商集成，以验证凭证和令牌。它可能管理用户会话或 JWT 令牌，并存储授权数据。设计明确提到使用云身份（CON-2）进行用户管理和 SSO，类似于案例研究的方法，但在这里它被封装在一个服务中。
可扩展性	案例研究中采用 CQRS 和微服务设计的主要动机是解决可扩展性和性能问题。通过将查询端与命令端分离，系统可以独立于写入操作扩展读取能力。在第三轮迭代中，设计明确增加了查询服务的复制和负载均衡以提高价格查询的吞吐量。Kafka 作为事件存储系统还允许根据需要扩展消费者（多个查询或导出处理器）。这种方法确保了来自外部系统查询价格的高读取负载可以通过单独向查询微服务添加资源来满足。其他可扩展性考虑因素包括避免共享状态（每个微服务都有自己的数据库）和使用无状态服务以促进水平扩展。	该架构考虑了每个微服务的独立扩展。例如，如果价格查询量激增，价格查询服务可以扩展到多个实例而不会影响其他实例。查询服务中加入的分布式缓存层通过减少数据库负载支持高读取吞吐量。此外，该架构在多个维度上考虑了扩展性：它使用 CQRS 来分配读写负载，并且可以利用多区域部署来分配负载并降低这些区域用户的延迟。决定将所有服务容器化并通过 Kubernetes 编排意味着平台可以根据需求自动扩展服务。

Deployability	The case study considers deploying all components on a cloud provider. In iteration 1, it was decided to keep all microservices and the event bus within a private network, accessible only through the API Gateway (enhancing security). By iteration 3, more concrete deployment tactics are defined: using a container orchestration service (likely a managed Kubernetes) to run and monitor the microservices, and setting up passive redundancy (a standby deployment in a second availability zone/region) to take over on failures. The event bus (Kafka) is a managed, durable service, removing the need to manage message brokers manually. Each microservice has its own database (e.g., a SQL DB for Command, NoSQL for Query) provided as managed cloud databases. Continuous deployment (CI/CD) is satisfied using a proprietary git-based platform with support for pipelines.	The architecture specifies a robust deployment and DevOps strategy. All services are packaged as containers and deployed on a container orchestration platform (Kubernetes) for consistency across environments. A continuous integration/continuous deployment pipeline builds images, runs tests, and promotes deployments through staging to production. Configuration is managed separately (Infrastructure as Code and a config repository) to enable reproducible, environment-specific deployments. For high availability, the system is deployed in multiple regions with automated failover: if the primary region fails, a secondary region's services and database replicas take over, using global DNS or load balancer routing. The design also leverages managed cloud services (for databases, identity, etc.) to offload infrastructure management.
Testability	The use of the Spring framework and its inversion of control was selected to facilitate unit testing of modules. A concern (CRN-7) was added to ensure all custom business logic components can be unit tested. This implies the design will include a suite of unit tests for services, controllers, etc., and likely some integration tests for the event flow. However, the case study description does not describe a full testing environment setup. No specific mention is made of test automation tools or frameworks beyond standard unit testing.	The design dedicates an entire infrastructure for testing. It uses containerized test environments via TestContainers to spin up ephemeral databases, Kafka brokers, and other dependencies for integration tests. There is a suite for contract testing to ensure that each service's APIs meet the expectations of its consumers (using consumer-driven contracts and schema validation). Mock versions of external systems are provided to test interactions without relying on live systems. The test infrastructure supports automated provisioning of these components to run integration tests in CI pipelines. Additionally, test data management tools reset and seed databases between tests to ensure isolation. All of this is integrated into the CI/CD pipeline, so tests run on each build and deployment, with reporting and failure alerts.

Table A. Design comparison between published case study and LLM solution

部署能力	本案例研究探讨了在云服务商部署所有组件的方案。在第一阶段迭代中，我们决定将所有微服务和事件总线部署在私有网络内，仅通过API网关访问（以增强安全性）。到第三阶段迭代时，具体部署策略更加明确：采用容器编排服务（可能是托管的Kubernetes）来运行和监控微服务，并设置被动冗余机制（在第二个可用区/区域部署备用系统以应对故障）。事件总线（Kafka）作为托管的持久化服务，省去了手动管理消息代理的需要。每个微服务都拥有专属数据库（例如命令系统使用SQL数据库，查询系统使用NoSQL数据库），这些数据库均由托管云数据库提供。持续集成/持续交付（CI/CD）通过支持管道的专有Git平台实现。	该架构方案制定了稳健的部署与DevOps策略。所有服务均采用容器化封装，并通过容器编排平台（Kubernetes）实现跨环境一致性部署。持续集成/持续交付管道负责构建镜像、执行测试，并通过预发布环境将应用部署至生产环境。配置管理采用基础设施即代码（IaC）与配置仓库分离的方式，确保部署过程可复现且环境适配性高。为保障高可用性，系统部署于多个区域并配备自动故障转移机制：当主区域发生故障时，次区域的服务与数据库副本将通过全局DNS或负载均衡器实现无缝接管。该架构还通过托管云服务（涵盖数据库、身份认证等模块）来分担基础设施管理任务。
可测试性	使用Spring框架及其控制反转是为了便于模块的单元测试。增加了一个关注点（CRN -7），以确保所有自定义业务逻辑组件都能进行单元测试。这意味着设计将包括一系列针对服务、控制器等的单元测试，以及可能的一些事件流的集成测试。然而，案例研究描述中并未提及完整的测试环境设置。除了标准单元测试外，没有具体提到测试自动化工具或框架。	该设计方案专门构建了完整的测试基础设施。通过TestContainers容器化测试环境，可快速部署临时数据库、Kafka代理等集成测试组件。系统还配备契约测试套件，确保各服务API符合消费者预期（使用（消费者驱动的合同与模式验证）。系统提供外部系统的模拟版本，无需依赖真实系统即可测试交互。测试基础设施支持自动化配置这些组件，以便在持续集成（CI）管道中运行集成测试。此外，测试数据管理工具会在测试间重置并初始化数据库，确保测试隔离性。所有这些功能都集成到CI/CD管道中，因此每次构建和部署时都会运行测试，并生成报告和故障警报。

表A. 已发表的案例研究与LLM解决方案的设计比较

Appendix B

The following table compares the solution generated by the LLM with the ones from the online solution.

Area	Ticketmaster Article	LLM generated solution
Service Granularity	Small set of coarse-grained services (API Gateway, Event, Search, Booking, Payment)	Fine-grained microservices: Auth, User, Event, Search, Inventory, Order, Payment, Ticket, Delivery, Notification, etc.
Seat Reservation Strategy	Prefers a distributed lock with TTL in Redis to prevent double-booking	Uses optimistic locking in the Inventory DB plus cache invalidation and event sourcing
Peak-Demand Mitigation	Suggests Server-Sent Events for live seat updates and an optional virtual waiting room built on WebSockets	No explicit real-time push or waiting-room mechanism; relies on caching and eventual consistency via MQ events
Search Synchronisation	Change Data Capture (CDC) pipeline from PostgreSQL to Elasticsearch to keep the index fresh in near real time	Event Service publishes “EventCreated” messages; Search Service consumes and indexes, Event-driven rather than CDC
Ticket Generation & Delivery	Out of scope—design stops once booking is confirmed.	Dedicated Ticket Service and Delivery Service with multi-channel strategy (email / mobile)
Security & Compliance	Assumes Stripe handles payment security; little depth on broader security.	Explicit PaymentSecurityManager, encryption, circuit-breaker around gateway, JWT/OAuth via external IdP
Data Stores	Single relational DB (PostgreSQL) plus Redis & Elasticsearch.	Dedicated DB per service, read-replicas, potential sharding and audit logs per domain

附录B

下表比较了由大型语言模型（LLM）生成的解决方案与在线解决方案的对比结果。

区域	票务系统文章	LLM生成的解决方案
服务粒度	一组小型粗粒度服务（API网关、事件处理、搜索、预订、支付）	细粒度微服务：认证、用户、事件、搜索、库存、订单、支付、工单、配送、通知等。
座位预订策略	在Redis中首选带有 TTL 的分布式锁，以防止双重预订	在库存数据库中采用乐观锁机制，同时结合缓存失效和事件溯源技术
缓解高峰需求	建议通过服务器发送事件实现实时座位更新，并可选配基于WebSockets构建的虚拟候补室	没有明确的实时推送或等候室机制；依赖于缓存和通过 MQ 事件实现的最终一致性
搜索同步	将数据捕获（CDC）流程从PostgreSQL迁移至Elasticsearch，以实现近乎实时的索引更新	事件服务发布“EventCreated”消息；搜索服务以事件驱动而非CDC方式进行消息消费与索引
票务生成与交付	超出范围——设计在预订确认后即告终止。	采用多渠道策略（电子邮件/手机）的专属票务服务与配送服务
安全&依从性	默认Stripe负责处理支付安全；对更广泛的安全问题探讨不足。	显式支付安全管理器、加密、网关断路器、通过外部 IdP 的 JWT /OAuth
数据存储	单一关系型数据库（PostgreSQL）搭配Redis与Elasticsearch。	按服务划分的专用数据库、读取副本、潜在的分片方案及每个域的审计日志

Table B. *Solution comparison*

表B. 溶液比较